

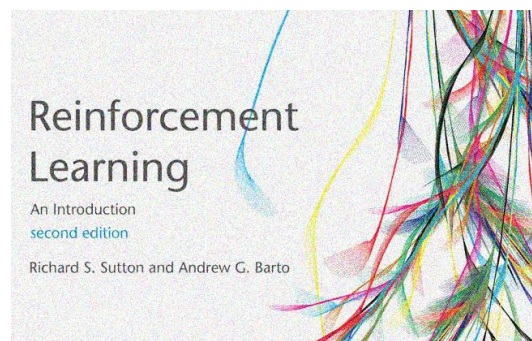


دانشگاه اصفهان

دانشکده مهندسی کامپیوتر - گروه هوش مصنوعی و رباتیک

گزارش تمرین اول - مفاهیم یادگیری تقویتی

MDP-BL-PI-VI



پدیدآورنده:

محمد امین کیانی

۴۰۴۳۶۴۴۰۰۸

دانشجوی کارشناسی ارشد، دانشکده‌ی کامپیوتر، دانشگاه اصفهان، اصفهان،
Aminkianiworkeng@gmail.com

استاد درس: دکتر کارشناس

نیمسال دوم تحصیلی ۱۴۰۴-۰۵

فهرست مطالب

مستندات.....	۴
۰- مسئله و تحلیل کلی آن:.....	۴
۱- بخش اول: MDP.....	۴
ربات جمع‌آوری کالا در انبار.....	۴
جدول احتمالات و گراف انتقال.....	۱۱
۲- بخش دوم: MRP.....	۱۵
ارتباط MDP و MRP با هم و policy evaluation.....	۱۵
سیاست فرضی غیرقطعی و غیر یکنواخت برای مثال بخش اول.....	۱۶
گراف و جدول MRP حاصل از سیاست بخش قبلی.....	۱۷
۳- بخش سوم: مونت کارلو.....	۲۱
دو اپیزود نمونه.....	۲۲
First-Visit Monte Carlo برای همه‌ی stateها.....	۲۲
Every-Visit Monte Carlo برای همه‌ی stateها.....	۲۳
بردار پاداش و ماتریس انتقال و حل دقیق Bellman.....	۲۵
آیا First-Visit و Every-Visit به $v(s)$ همگرا می‌شوند؟.....	۲۷
۴- بخش چهارم: VI and PI.....	۳۳
الگوریتم Synchronous Policy Iteration.....	۳۳
الگوریتم Synchronous Value Iteration.....	۳۶
معادله‌ی Bellman optimality.....	۳۷
سیاست greedy بر اساس V_2	۳۸
فرض کنیم V_2 بهینه است، سیاست بهینه چیست؟.....	۳۹
چرا سیاست سؤال ۱ بدتر از سیاست اولیه نیست؟ (همین استدلال برای سؤال ۴).....	۳۹
در این MDP تغییر rewardها اثر بیشتری دارد یا تغییر transitionها؟.....	۴۰

- ۵- بخش پنجم : **Horizon** ۴۱
- فرمال سازی کامل مدل ۴۱
- رابطه بازگشتی برنامه ریزی پویا برای افق محدود ۴۳
- کمترین افق لازم برای رسیدن از $S_0=4$ به ۱۲ ۴۴
- عمل اول بهینه برای $H=3,6,8,10$ ۴۵
- مقایسه‌ی دو سیاست مرجع: "فقط برداشت" و "فقط تأمین تا terminal" ۴۶
- آیا مسیری بهینه وجود دارد که هر دو عمل برداشت و تأمین را داشته باشد؟ ۴۷
- اگر پاداش انتقال ۱۱ به ۱۲ به جای ۷۰ برابر R باشد، مرز سیاست‌ها چگونه جابه‌جا می‌شود؟ ۴۸
- در حالت افق بی‌نهایت تخفیف‌دار، آیا ممکن است برای بعضی ۷ها سیاست بهینه هرگز به terminal نرسد؟ ۵۰
- چرا discounting لزوماً معادل finite horizon نیست؟ ۵۱
- ۶- بخش ششم : **Reward Misspecification** ۵۲
- مثال ترافیکی مقاله چه می‌گوید و ربط آن به proxy reward و true reward در چیست؟ ۵۲
- چرا با افزایش توان عامل، proxy reward بالا می‌رود ولی true reward پایین می‌آید؟ ۵۳
- phase transition یعنی چه و در این مقاله چه نقشی دارد؟ ۵۵
- این مسئله بیشتر به misweighting، ontological یا scope نزدیک است؟ ۵۵
- چرا اصلاً reward hacking رخ می‌دهد؟ ۵۶
- دو ایده برای proxy reward بهتر ۵۷
- مسئله‌ی anomaly detection در این مقاله چیست؟ ۵۷
- چرا trend extrapolation به تنهایی کافی نیست؟ ۶۱
- ۷- مراجع ۶۳

مستندات

۰- مسئله و تحلیل کلی آن:

مباحث این تمرین دقیقاً روی محورهای زیر است:

MDP, MRP, Monte Carlo, Policy Iteration / Value Iteration, Horizon, و در

ادامه بخش Reward Misspecification بر پایه‌ی مقاله.

در Sutton, یک MDP با دنباله‌ی $S_0, A_0, R_1, S_1, \dots$ و تابع دینامیک مشترک $p(s', r | s, a)$ تعریف می‌شود و «Markov property» یعنی state باید تمام اطلاعات لازم از گذشته را برای پیش‌بینی آینده خلاصه کند؛ یعنی اگر دو history به یک state نگاشت شوند، باید توزیع آینده‌شان یکی باشد.

۱- بخش اول: MDP

ربات جمع‌آوری کالا در انبار

ربات در یک انبار بسته‌بندی، بسته‌ها را جمع می‌کند و باتری قابل شارژ دارد. تصمیم‌های سطح بالا فقط بر اساس وضعیت فعلی باتری و شرایط محیط گرفته می‌شوند.

مجموعه‌ی حالت‌ها

فرض می‌کنیم state به صورت گسسته تعریف شده است:

• H = باتری بالا، مسیر حرکت باز

• M = باتری متوسط، مسیر نیمه‌شلوغ

• L = باتری پایین، مسیر شلوغ و پرریسک

پس: $S = \{H, M, L\}$

مجموعه‌ی اعمال

- در H : search, wait
- در M : search, wait, recharge
- در L : search, recharge

پس در این مثال چند state داریم که بیش از یک action دارند.

جدول توزیع توأم $p(s',r|s,a)$

در این جا تابع دینامیک را با احتمال‌های joint state-reward می‌نویسیم؛ دقیقاً همان فرم Sutton در معادله‌ی (۳.۲). همچنین از این توزیع می‌توان $p(s'|s,a)$ ، $r(s,a)$ و $r(s,a,s')$ را طبق (۳.۴) تا (۳.۶) به‌دست آورد.

برای حالت H

(s)	(a)	(s')	(r)	$p(s',r s,a)$
H	search	H	+3	0.20
H	search	M	+2	0.40
H	search	L	0	0.25
H	search	L	-1	0.15
H	wait	H	0	0.80
H	wait	M	0	0.20

برای حالت M

(s)	(a)	(s')	(r)	$p(s',r s,a)$
M	search	H	+3	0.10
M	search	M	+2	0.35
M	search	L	0	0.20
M	search	L	-2	0.35
M	wait	M	0	0.65
M	wait	L	0	0.35
M	recharge	H	0	0.90
M	recharge	M	0	0.10

برای حالت L

(s)	(a)	(s')	(r)	$p(s',r s,a)$
L	search	M	-1	0.25
L	search	L	-2	0.45
L	search	L	-4	0.30
L	recharge	M	0	0.70
L	recharge	H	0	0.30

بررسی شرط‌های مسئله

شرط ۱: حداقل سه وضعیت

بله، سه state داریم: H, M, L.

شرط ۲: بعضی state ها بیش از یک action دارند.

بله:

• در H : دو action داریم.

• در M : سه action داریم.

• در L : دو action داریم.

شرط ۳: انتقال غیرقطعی و با توزیع غیر یکنواخت.

بله. مثلاً در H با search action :

$$p(H, +3 | H, \text{search}) = 0.20, \quad p(M, +2 | H, \text{search}) = 0.40, \quad p(L, 0 | H, \text{search}) = 0.25, \\ p(L, -1 | H, \text{search}) = 0.15$$

اینجا هم stochastic (تصادفی) است و هم توزیع یکنواخت ندارد.

شرط ۴: برای یک action و یک next state، حداقل دو reward متفاوت باشد.

بله. مثلاً در M state با search action و رسیدن به L، دو reward متفاوت داریم:

$$p(L, 0 | M, \text{search}) = 0.20, \quad p(L, -2 | M, \text{search}) = 0.35$$

و در L state با search action و رسیدن به L نیز دو reward متفاوت داریم:

$$p(L, -2 | L, \text{search}) = 0.45, \quad p(L, -4 | L, \text{search}) = 0.30$$

- آیا Markov Property برقرار است؟

در دنیای واقعی، به صورت خام: نه، دقیقاً برقرار نیست

اگر state را فقط H/M/L بگیریم، این خلاصه سازی معمولاً برای محیط واقعی کافی نیست؛ چون آینده فقط به باتری وابسته نیست. مثلاً این عوامل هم روی آینده اثر می گذارند:

- تراکم مسیرها
- تعداد بسته های باقی مانده
- وضعیت دقیق مکان ربات
- زمان شیفِت کاری
- دمای باتری یا سلامت باتری
- میزان شلوغی انبار

پس دو history متفاوت ممکن است به یک state ظاهری مثل M برسند، اما توزیع آینده شان یکسان نباشد. این دقیقاً نقض Markov property است؛ چون Sutton می گوید state باید تمام اطلاعات مؤثر بر آینده را در خود داشته باشد.

- چه فرضی لازم است تا Markov شود؟

باید فرض کنیم state ما یک representation کافی و کامل از وضعیت محیط است؛ یعنی state شامل همه ی متغیرهای مؤثر باشد.

در این مثال، فرض Markov را این طور اعمال می کنیم:

st=(battery level, aisle congestion, package density, location zone)

اما برای ساده سازی مسئله، این اطلاعات را به سه کلاس H, M, L فشرده کرده ایم، با این فرض که این فشرده سازی هنوز sufficient state است. به زبان کتاب، state باید

«information about all aspects of the past interaction that make a difference for the future» را نگه دارد .

این مثال یک MDP واقعی برای ربات انبار است که سه state دارد، در بعضی state ها چند action ممکن است، انتقال ها stochastic و غیر یکنواخت اند، و حتی برای یک action و یک next state چند reward مختلف وجود دارد. اما اگر state را فقط در حد H/M/L بگیریم، در دنیای واقعی Markov property دقیقاً برقرار نیست؛ برای برقرار شدن آن باید state را به صورت یک **state کامل و sufficient** تعریف کنیم که همه ی متغیرهای مؤثر بر آینده را در بر بگیرد. Sutton و Barto دقیقاً MDP را بر پایه ی همین state Markov و دینامیک $p(s',r|s,a)$ بنا می کنند.

چرا از یک state به یک state دیگر چند پاداش مختلف داریم؟

در RL، محیط فقط state بعدی را تصادفی انتخاب نمی کند؛ گاهی reward هم تصادفی است.

به فرض ربات در وضعیت M (باتری متوسط) است و $action = search$ را انتخاب می کند.

ربات دنبال بسته می رود اما مثلاً دو اتفاق ممکن است بیفتد:

حالت اول: ربات بسته ای پیدا نمی کند، ولی فقط باتری کم می شود.

پس:

• $next\ state = L$

• $reward = 0$

حالت دوم: ربات علاوه بر اینکه باتری اش کم می شود، به خاطر برخورد یا اشتباه، جریمه هم می شود.

پس:

• $\text{next state} = L$

• $\text{reward} = -2$

next state در هر دو یکی است (L) اما reward فرق دارد. پس اینجا environment ذاتاً stochastic یا تصادفی است.

اعداد reward را چگونه بدست آوردیم؟

طبق صورت مسئله، لازم نیست reward واقعی باشد و می توان پارامتر یا عددی فرضی در نظر گرفته شود. پس ما designer محیط هستیم.

مثلاً:

action	reward
پیدا کردن بسته	+3
کار مفید	+2
مصرف باتری	-1
حرکت پرهزینه	-2
خرابی یا ریسک بالا	-4

چون هدف ربات جمع آوری بسته است. پس rewardها براساس «مطلوب بودن» طراحی می شوند.

Reward = صفر یعنی:

• نه سود و نه ضرر (ربات چیزی پیدا نکرده و فقط زمان گذشته)

چون دنیای واقعی uniform نیست و ربات معمولاً عملکرد متوسط دارد و خیلی کم عالی عمل می‌کند و بعضی وقت‌ها خراب می‌کند، پس احتمال‌ها طبیعی نیست که برابر باشند ولی مجموع probability هر حالت بعدی ناشی از حالت فعلی باید برابر با ۱ باشد حتی اگر احتمال رخداد برابری نداشته باشند.

جدول احتمالات و گراف انتقال

Sutton تأکید می‌کند که در RL بهتر است دینامیک را با توزیع توأم $p(s', r | s, a)$ بنویسیم، نه فقط با $p(s' | s, a)$ چون reward و state بعدی در عمل می‌توانند با هم و به صورت تصادفی تعیین شوند.

(s)	(a)	(s')	$p(s' s, a)$	$r(s, a, s')$
H	search	H	۰.۲۰	+۳
H	search	M	۰.۴۰	+۲
H	search	L	۰.۴۰	-۰.۳۷۵
M	search	H	۰.۱۰	+۳
M	search	M	۰.۳۵	+۲
M	search	L	۰.۵۵	-۱.۲۷۳
L	search	M	۰.۲۵	-۱
L	search	L	۰.۷۵	-۲.۸
H	wait	H	۰.۸۰	۰
H	wait	M	۰.۲۰	۰

M	wait	M	۰.۶۵	۰
M	wait	L	۰.۳۵	۰
L	recharge	M	۰.۷۰	۰
L	recharge	H	۰.۳۰	۰
M	recharge	H	۰.۹۰	۰
M	recharge	M	۰.۱۰	۰

– از جدول joint چگونه $p(s'|s,a)$ بدست می آید؟

با جمع کردن روی rewardها:

$$p(s' | s, a) = \sum_r p(s', r | s, a)$$

مثلاً برای H, search :

$$p(H|H,search)=۰.۲۰, p(M|H,search)=۰.۴۰, p(L|H,search)=۰.۲۵ + ۰.۱۵=۰.۴۰$$

پس:

$$p(\cdot|H,search)=\{۰.۲۰, ۰.۴۰, ۰.۴۰\}$$

این یعنی یک transition distribution غیرقطعی و غیر یکنواخت است: چون سه outcome دارد، اما احتمالها برابر نیستند.

- از جدول joint چگونه $r(s,a,s')$ به دست می آید؟

اگر بخواهیم reward مورد انتظار برای یک next-state خاص را بنویسیم، به صورت زیر است:

$$r(s, a, s') = \frac{\sum_r r p(s', r | s, a)}{p(s' | s, a)}$$

مثلاً برای $s'=L$ و $H, search$:

$$p(L|H,search)=0.25+0.15=0.40$$

و reward مورد انتظارِ شرطی:

$$r(H,search,L)=((0)*0.25)+((-1)*0.15)/0.40=-0.375$$

یعنی اگر به L برسیم، میانگین reward در آن outcome برابر -0.375 است. همین کار را می توان برای بقیه ی سطرها هم انجام داد...

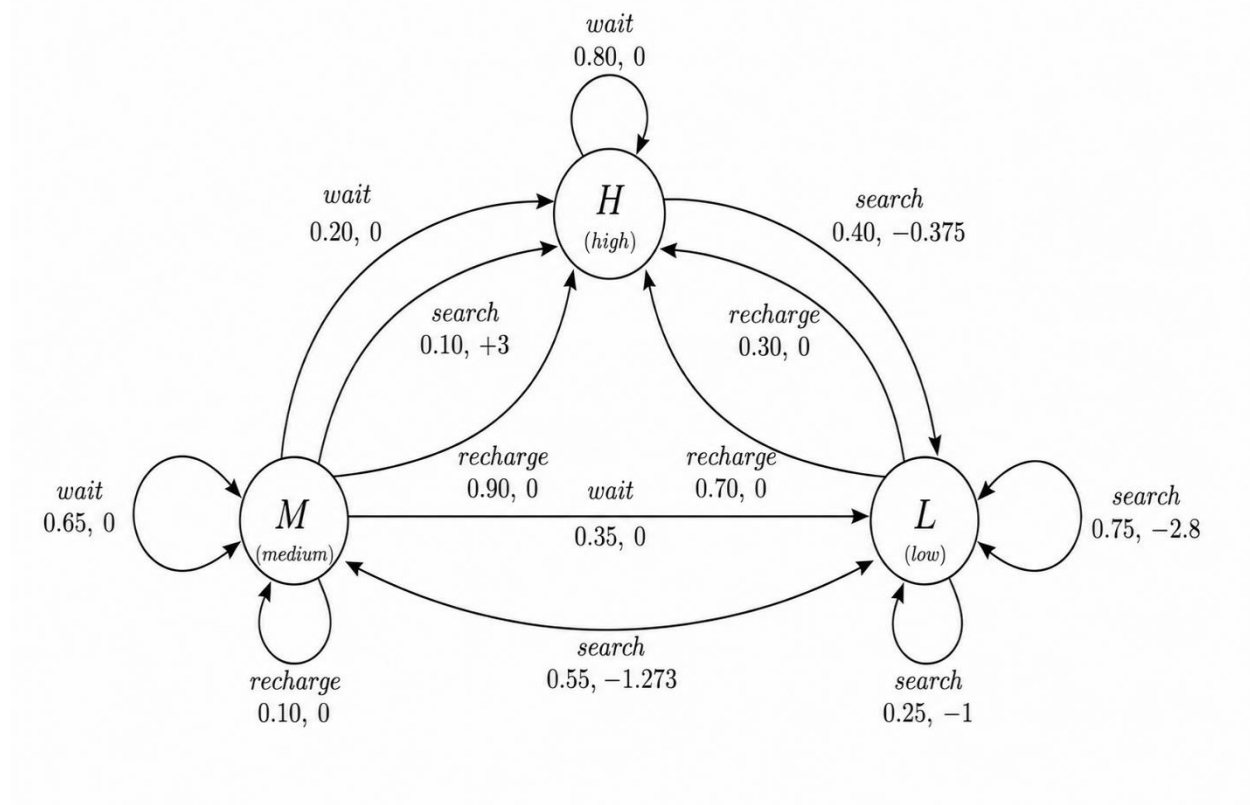
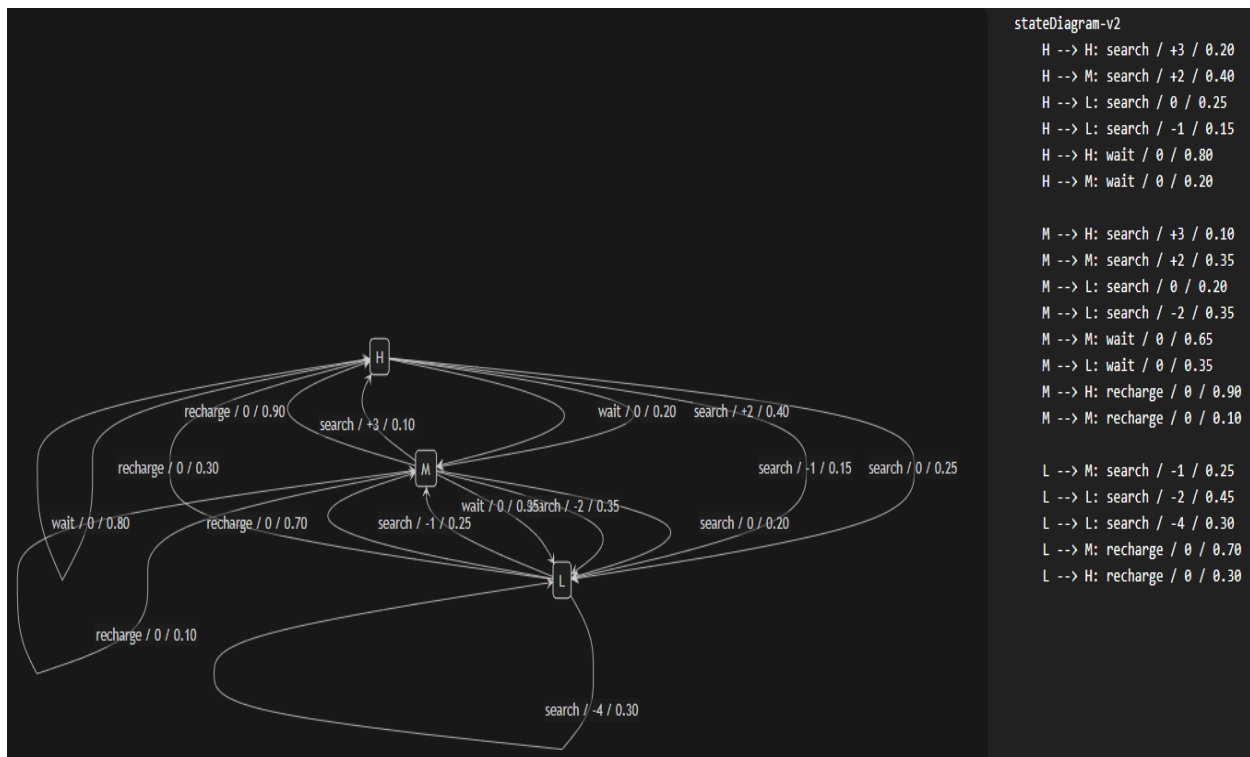
- گراف انتقال متناظر

هر یال، یک outcome از (s,a) است و روی آن action / reward / probability می نویسیم.
پس:

* سه دایره برای H,M,L می کشیم.

* از هر حالت، یال های مربوط به action ها را رسم می کنیم.

* روی هر یال، reward و probability را هم می نویسیم.



۲- بخش دوم: MRP

در کتاب Sutton، MRP در اصل همان **MDP بدون action** است؛ یعنی وقتی سیاست π را ثابت می‌کنیم، دیگر «انتخاب عمل» جداگانه نداریم و دینامیک محیط تحت همان سیاست به یک فرایند مارکوفی پاداش‌دار تبدیل می‌شود. همچنین سیاست در RL به صورت نگاشت $state$ به توزیع احتمالی $action$ تعریف می‌شود و $value$ function هم بر حسب $expected$ return و نسبت به یک $policy$ تعریف می‌شود.

ارتباط MDP و MRP با هم و $policy$ evaluation

اگر یک MDP داشته باشیم:

$$\mathcal{M} = (S, A, p(s', r | s, a), \gamma)$$

و یک سیاست ثابت $\pi(a|s)$ را با میانگین‌گیری روی $action$ روی آن اعمال کنیم، آن وقت از دید $state$ فقط، یک MRP به دست می‌آید:

$$\mathcal{M}_\pi = (S, p_\pi(s', r | s), \gamma)$$

$$p_\pi(s', r | s) = \sum_a \pi(a | s) p(s', r | s, a)$$

یعنی $action$ به صورت فیزیکی حذف نمی‌شوند ولی اثرشان میانگین‌گیری می‌شود و در $transition$ های MRP جذب می‌شود. این همان ایده‌ای است که Sutton در بخش $policy$ evaluation می‌گوید: برای یک $policy$ ثابت، ارزش $state$ با حل $Bellman$ expectation equation به دست می‌آید. پس:

• **MDP** = هم $state$ داریم هم $action$

• **MRP** = $action$ ها را با $policy$ ثابت «فرو می‌ریزیم» و فقط $state$ و $reward$ و $transition$ می‌ماند

• **policy evaluation** = محاسبه‌ی v_π برای همین MRP حاصل از π

معادله‌ی ارزیابی سیاست در فرم Sutton :

$$v_{\pi}(s) = \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi}(s')]$$

و اگر بخواهیم به فرم MRP بنویسیم:

$$v_{\pi}(s) = \sum_{s', r} p_{\pi}(s', r | s) [r + \gamma v_{\pi}(s')]$$

تابع ارزش حالت برای policy evaluation از معادله‌ی امید بلمن پیروی می‌کند یا به فرم ماتریسی $v = r_{\pi} + \gamma P_{\pi} v$ می‌شود که این دقیقاً همان پل بین policy evaluation و MRP است.

سیاست فرضی غیرقطعی و غیر یکنواخت برای مثال بخش اول

من برای مثال سه حالتی بخش قبلی، این policy را پیشنهاد می‌کنم زیرا:

state	search	wait	recharge
H	۰.۷	۰.۳	۰
M	۰.۵	۰.۲	۰.۳
L	۰.۴	۰	۰.۶

۱. غیرقطعی است: در هر state فقط یک action با احتمال ۱ انتخاب نمی‌شود.

۲. غیر یکنواخت است: احتمال‌ها برابر نیستند.

۳. با منطق مثال هم سازگار است:

○ سیاست: وقتی state خوب است، search محتمل‌تر است ولی وقتی ضعیف‌تر باشد،

recharge هم وارد تصمیم می‌شود.

گراف و جدول MRP حاصل از سیاست بخش قبلی
 اکنون از MDP بخش اول، با ثابت کردن policy بالا، یک MRP می‌سازیم.

فرمول محاسبه‌ی MRP

ضرب احتمال اینکه policy آن action را انتخاب کند و احتمال رسیدن به state بعدی:

$$p_{\pi}(s' | s) = \sum_a \pi(a | s) p(s' | s, a)$$

و اگر بخواهیم reward روی هر یال MRP را بنویسیم (امید شرطی):

$$r_{\pi}(s, s') = \frac{\sum_a \pi(a | s) \sum_r r p(s', r | s, a)}{p_{\pi}(s' | s)}$$

همچنین reward متوسط هر state هم برابر است با:

$$r_{\pi}(s) = \sum_a \pi(a | s) \sum_{s', r} r p(s', r | s, a)$$

نتیجه‌ی محاسبه برای مثال ما

از $H = \text{state}$

	$:H \rightarrow H \bullet$
$p_{\pi}(H H) = 0.7(0.20) + 0.3(0.80) = 0.38$	
$r_{\pi}(H, H) = \frac{0.7(0.20)(3) + 0.3(0.80)(0)}{0.38} = 1.105$	
	$:H \rightarrow M \bullet$
$p_{\pi}(M H) = 0.7(0.40) + 0.3(0.20) = 0.34$	
$r_{\pi}(H, M) = \frac{0.7(0.40)(2) + 0.3(0.20)(0)}{0.34} = 1.647$	
	$:H \rightarrow L \bullet$
$p_{\pi}(L H) = 0.7(0.40) + 0.3(0) = 0.28$	
$r_{\pi}(H, L) = \frac{0.7(0.40)(-0.375) + 0.3(0)}{0.28} = 0$	
پس:	
$r_{\pi}(H) = 0.98$	

از **M = state**

• $M \rightarrow H$:

$$p_{\pi}(H | M) = 0.5(0.10) + 0.3(0.90) = 0.32$$

$$r_{\pi}(M, H) = \frac{0.5(0.10)(3) + 0.3(0.90)(0)}{0.32} = 0.469$$

• $M \rightarrow M$:

$$p_{\pi}(M | M) = 0.5(0.35) + 0.2(0.65) + 0.3(0.10) = 0.335$$

$$r_{\pi}(M, M) = \frac{0.5(0.35)(2) + 0.2(0.65)(0) + 0.3(0.10)(0)}{0.335} = 1.045$$

• $M \rightarrow L$:

$$p_{\pi}(L | M) = 0.5(0.55) + 0.2(0.35) = 0.345$$

$$r_{\pi}(M, L) = \frac{0.5[(0.20)(0) + (0.35)(-2)]}{0.345} = -1.014$$

و reward متوسط state:

$$r_{\pi}(M) = 0.15$$

از **L = state**

• $L \rightarrow H$:

$$p_{\pi}(H | L) = 0.4(0) + 0.6(0.30) = 0.18$$

$$r_{\pi}(L, H) = 0$$

• $L \rightarrow M$:

$$p_{\pi}(M | L) = 0.4(0.25) + 0.6(0.70) = 0.52$$

$$r_{\pi}(L, M) = \frac{0.4(0.25)(-1) + 0.6(0.70)(0)}{0.52} = -0.192$$

• $L \rightarrow L$:

$$p_{\pi}(L | L) = 0.4(0.75) + 0.6(0) = 0.30$$

$$r_{\pi}(L, L) = \frac{0.4[(0.45)(-2) + (0.30)(-4)]}{0.30} = -2.8$$

و reward متوسط state:

$$r_{\pi}(L) = -0.94$$

جدول نهایی یال‌ها برای گراف MRP

- راه حل مثلاً برای H (بقیه هم به همین صورت):

$$r_{\pi}(H) = 0.7(0.20 \cdot 3 + 0.40 \cdot 2 + 0.25 \cdot 0 + 0.15 \cdot (-1)) + 0.3 \cdot 0 = 0.875$$

$$P_{\pi}(H, H) = 0.7(0.20) + 0.3(0.80) = 0.380$$

Transition matrix (H,M,L):

$$P_{\pi} = \begin{bmatrix} 0.38 & 0.34 & 0.28 \\ 0.32 & 0.335 & 0.345 \\ 0.18 & 0.52 & 0.30 \end{bmatrix}$$

مثلاً:

$$P(H \rightarrow M) = 0.34$$

Expected reward vector:

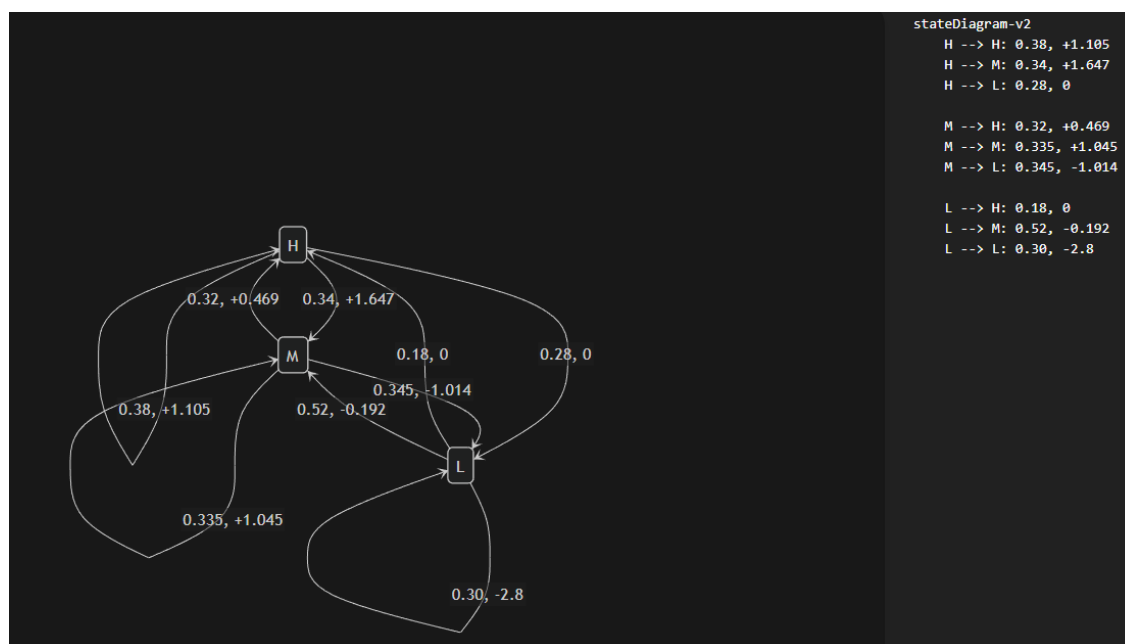
$$r_{\pi} = \begin{bmatrix} 0.875 \\ 0.15 \\ -0.94 \end{bmatrix}$$

یعنی:

اگر اکنون در H باشیم، میانگین reward برابر ۰.۸۷۵ است.

from	to	probability	reward
H	H	۰.۳۸	+۱.۱۰۵
H	M	۰.۳۴	+۱.۶۴۷
H	L	۰.۲۸	۰

M	H	۰.۳۲	+۰.۴۶۹
M	M	۰.۳۳۵	+۱.۰۴۵
M	L	۰.۳۴۵	-۱.۰۱۴
L	H	۰.۱۸	۰
L	M	۰.۵۲	-۰.۱۹۲
L	L	۰.۳۰	-۲.۸



MDP وقتی policy ثابت π داشته باشد، به MRP تبدیل می‌شود؛ یعنی action ها در transition های induced-by-policy جمع می‌شوند و یک فرایند فقط با حالت به‌دست می‌آید. این دقیقاً مبنای policy evaluation است، چون در policy evaluation هدف محاسبه v_π برای همان MRP ناشی از π است. فرم کلی امید شرطی:

$$E[X|A] = \frac{\sum xP(x, A)}{P(A)}$$

۳- بخش سوم: مونت کارلو

برای اینکه این بخش بدون تخفیف معنی دار و قابل حل باشد، یک نسخه‌ی episodic از همان MRP سه‌حالت را در نظر می‌گیریم و یک حالت پایانی T هم اضافه می‌کنیم تا با تعریف Monte Carlo در Sutton سازگار است زیرا در فصل ۵ مونت کارلو را برای **episodic tasks** تعریف می‌کند و ارزش هر state را با میانگین **return** های کامل برآورد می‌کند. همچنین First-Visit و Every-Visit MC به ترتیب میانگین return اولین بازدید و همه‌ی بازدیدها را می‌گیرند و با زیاد شدن تعداد بازدیدها به $v_{\pi}(S)$ همگرا می‌شوند.

مدل MRP برای حل این بخش:

Monte Carlo باید اپیزود داشته باشد اما MRP بخش دوم continuing بود و terminal نداشت. پس برای استفاده از MC، باید همان MRP را **episodic** کنیم.

پس یک state پایانی اضافه می‌کنیم:

$$\text{terminal} = T$$

و فرض می‌کنیم همانند Sutton که برای MC فرض episodic بودن را لازم می‌داند:

- هر state با احتمال کمی می‌تواند terminate شود.
- مثلاً اگر transition معمولی انجام نشود، episode تمام شود.

حالت‌های غیرپایانی: $S = \{H, M, L\}$ و حالت پایانی: T با $v(T) = 0$

گذارها و پاداش‌ها:

هر اپیزود فقط یک **sample path** از همان جدول بخش اول است؛ یعنی برای هر گام، فقط یکی از خروجی‌های مجاز همان state/transition را می‌توان انتخاب کرد.

دو اپیزود نمونه

دو اپیزود نمونه‌ی حداقل ۴-مرحله‌ای می‌سازیم که با این MRP سازگار باشند:

اپیزود ۱

$$H + 3 \rightarrow H + 0 \rightarrow H + 0 \rightarrow M + 2 \rightarrow M + 0 \rightarrow T$$

$$H \xrightarrow{\text{search},+3} H \xrightarrow{\text{wait},0} H \xrightarrow{\text{wait},0} M \xrightarrow{\text{search},+2} M$$

اپیزود ۲

$$L - 1 \rightarrow M + 0 \rightarrow H + 3 \rightarrow H + 0 \rightarrow M + 0 \rightarrow T$$

$$L \xrightarrow{\text{search},-1} M \xrightarrow{\text{recharge},0} H \xrightarrow{\text{search},+3} H \xrightarrow{\text{wait},0} M$$

First-Visit Monte Carlo برای همه‌ی stateها

return هر state را از همان الگوریتم اولین بازدید در هر episode حساب می‌کنیم.

اپیزود ۱

• اولین بازدید از H در $t=0$:

$$G(H)=3+0+0+2+0=5$$

• اولین بازدید از M در $t=3$:

$$G(M)=2+0=2$$

• اولین بازدید از L در این اپیزود اتفاق نمی‌افتد :

$$G(L)=0$$

اپیزود ۲

- اولین بازدید از L در $t=0$:

$$G(M) = -1 + 0 + 3 + 0 + 0 = 2$$

- اولین بازدید از M در $t=1$:

$$G(L) = 0 + 3 + 0 + 0 = 3$$

- اولین بازدید از H در $t=2$:

$$G(H) = 3 + 0 + 0 = 3$$

برآورد First-Visit

$$V_{FV}(H) = (5 + 3) / 2 = 4$$

$$V_{FV}(M) = (2 + 3) / 2 = 2.5$$

$$V_{FV}(L) = (2) / 1 = 2$$

Every-Visit Monte Carlo برای همه‌ی stateها

اینجا همه‌ی بازدیدها را در نظر می‌گیریم.

اپیزود ۱

- بازدید از H در $t=0$:

$$G(H) = 3 + 0 + 0 + 2 + 0 = 5$$

- بازدید دوباره از H در $t=1$:

$$G(M) = 0 + 0 + 2 + 0 = 2$$

- بازدید دوباره از H در $t=2$:

$$G(L) = 0 + 2 + 0 = 2$$

- بازدید از M در $t=3$:

$$G(H) = 2$$

- بازدید دوباره از M در $t=4$:

$$G(H) = 0$$

اپیزود ۲

- بازدید از L در $t=0$:

$$G(M) = -1 + 0 + 3 + 0 + 0 = 2$$

- بازدید از M در $t=1$:

$$G(L) = 0 + 3 + 0 + 0 = 3$$

- بازدید از H در $t=2$:

$$G(H) = 3 + 0 + 0 = 3$$

- بازدید دوباره از H در $t=3$:

$$G(M) = 0 + 0 = 0$$

- بازدید دوباره از M در $t=4$:

$$G(M) = 0$$

برآورد Every-Visit:

$$V_{FV}(H) = (5 + 2 + 2 + 3 + 0) / 5 = 2.4$$

$$V_{FV}(M) = (2 + 0 + 3 + 0) / 4 = 1.25$$

$$V_{FV}(L) = 2$$

بردار پاداش و ماتریس انتقال و حل دقیق Bellman

اینجا باید مقادیر بدست آمده از MRP بخش دوم را بنویسیم، یعنی همان MRP القاشده از سیاست:

بردار پاداش مورد انتظار

$$r_{\pi} = \begin{bmatrix} 0.875 \\ 0.15 \\ -0.94 \end{bmatrix}$$

ماتریس انتقال

$$P_{\pi} = \begin{bmatrix} 0.38 & 0.34 & 0.28 \\ 0.32 & 0.335 & 0.345 \\ 0.18 & 0.52 & 0.30 \end{bmatrix}$$

در بخش دوم، action ها با policy حذف شده اند و برای هر state باید روی action ها میانگین بگیریم. مثلاً برای H :

$$r_{\pi}(H) = 0.7(0.2 * 3 + 0.4 * 2 + 0.25 * 0 + 0.15 * (-1)) + 0.3 * 0 = 0.875$$

و برای انتقال ها هم:

$$p_{\pi}(s'|s) = \sum_a \pi(a|s) p(s'|s, a)$$

که P_{π} و r_{π} دقیقاً از همان جدول بخش اول و همان policy بخش دوم به دست آمده اند.

معادله بلمن

چون بدون تخفیف است یعنی $\gamma=1$:

$$(I - P_{\pi})v = r_{\pi} \text{ یا } v = r_{\pi} + P_{\pi}v$$

در چنین حالتی، چون P_{π} یک ماتریس stochastic است، معمولاً eigenvalue برابر ۱ دارد،
یعنی: $\det(I - P_{\pi}) = 0$

پس حل مستقیم یکتای فینیت برای v وجود ندارد مگر اینکه یکی از این دو مورد را اضافه کنیم:

۱. یک discount factor با $\gamma < 1$

۲. یک state پایانی و episodic کردن مسئله

پس برای مقادیر بخش دوم با مدل فعلی بدون تخفیف و بدون terminal، مقدار دقیق تابع ارزش-حالت به صورت یکتای فینیت از معادله بلمن به دست نمی آید.

دستگاه معادلات

$$v(H) = 0.875 + 0.38v(H) + 0.34v(M) + 0.28v(L)$$

$$v(M) = 0.15 + 0.32v(H) + 0.335v(M) + 0.345v(L)$$

$$v(L) = -0.94 + 0.18v(H) + 0.52v(M) + 0.30v(L)$$

مقایسه‌ی سؤال ۲ و ۳ با سؤال ۴

• First-Visit و Every-Visit که بالا حساب کردیم، فقط برآورد نمونه‌ای از return هستند .

• سؤال ۴ می‌خواهد exact value بدهد اما در این MRP ادامه‌دار بدون تخفیف طبق بخش دوم، به صورت یکتا تعریف نمی‌شود .

• MC ها با دو اپیزود فقط تخمین‌اند و طبیعی است که با هم فرق داشته باشند .

آیا First-Visit و Every-Visit به $v(s)$ همگرا می‌شوند؟

بله، اما فقط تحت شرط‌های اصلی زیر:

۱. داده‌ها باید از همان policy ثابت تولید شوند .
۲. return ها باید finite باشند .
۳. هر state باید در طول زمان به تعداد کافی زیاد دیده شود .
۴. برای MC کلاسیک، معمولاً task باید episodic باشد و episode ها با احتمال ۱ terminate شوند .

اما اگر همان **MRP ساخته شده از همین دو اپیزود** را بنویسیم که خاتمه دارند:

۱- ماتریس انتقال P

از روی دو اپیزود، تعداد خروجی‌های مشاهده شده را می‌شماریم.

از حالت H

در کل از H پنج بار خارج شده‌ایم:

• $H \rightarrow H$ سه بار

• $H \rightarrow M$ دو بار

پس:

$$P(H \rightarrow H) = 3/5 = 0.6$$

$$P(H \rightarrow M) = 2/5 = 0.4$$

از حالت M

از M چهار بار خارج شده‌ایم:

• $M \rightarrow H$ یک بار

• $M \rightarrow M$ یک بار

• $M \rightarrow T$ دو بار

پس:

$$P(M \rightarrow H) = 1/4 = 0.25$$

$$P(M \rightarrow M) = 1/4 = 0.25$$

$$P(M \rightarrow T) = 2/4 = 0.5$$

از حالت **L**

فقط یک بار دیده شده:

• $L \rightarrow M$ با احتمال ۱

پس:

$$P(L \rightarrow M) = 1$$

حالت پایانی **T**

$$P(T \rightarrow T) = 1$$

۲- بردار پاداش مورد انتظار r

حالا reward مورد انتظار هر state را از میانگین rewardهای خروجی همان state می‌گیریم.

برای H

rewardهای خروجی از H در این دو اپیزود:

• 3,0,0,3,0

پس:

$$r(H) = (3+0+0+3+0)/5 = 6/5 = 1.2$$

برای M

reward های خروجی از M :

• 2,0,0,0

پس:

$$r(M) = 2/4 = 0.5$$

برای L

Reward خروجی از L :

• -1

پس:

$$r(L) = -1$$

برای T

$$r(T) = 0$$

$$r = \begin{bmatrix} 1.2 \\ 0.5 \\ -1 \\ 0 \end{bmatrix}$$
$$P = \begin{bmatrix} 0.6 & 0.4 & 0 & 0 \\ 0.25 & 0.25 & 0 & 0.5 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$
$$(H, M, L, T)$$

۳- حل مستقیم معادله بلمن

$$v = r + Pv$$

$$v(s) = r(s) + \sum_{s'} P(s, s')v(s')$$

با $v(T) = 0$ داریم:

برای L

$$v(L) = -1 + v(M)$$

برای M

$$v(M) = 0.5 + 0.25v(H) + 0.25v(M) + 0.5v(T)$$

و چون $v(T) = 0$:

$$v(M) = 0.5 + 0.25v(H) + 0.25v(M)$$

برای H

$$v(H) = 1.2 + 0.6v(H) + 0.4v(M)$$

حل دستگاه

از معادله M :

$$v(M) - 0.25v(M) = 0.5 + 0.25v(H)$$

$$0.75v(M) = 0.5 + 0.25v(H)$$

$$v(M) = \frac{2}{3} + \frac{1}{3}v(H)$$

از معادله H :

$$v(H) - 0.6v(H) = 1.2 + 0.4v(M)$$

$$0.4v(H) = 1.2 + 0.4v(M)$$

$$v(H) = 3 + v(M)$$

حالا جایگذاری:

$$v(M) = \frac{2}{3} + \frac{1}{3}(3 + v(M))$$

$$v(M) = \frac{2}{3} + 1 + \frac{1}{3}v(M)$$

$$v(M) - \frac{1}{3}v(M) = \frac{5}{3}$$

$$\frac{2}{3}v(M) = \frac{5}{3}$$

$$v(M) = \frac{5}{2} = 2.5$$

$$v(H) = 5.5, \quad v(M) = 2.5, \quad v(L) = 1.5, \quad v(T) = 0$$

۴- مقایسه و تحلیل First-Visit / Every-Visit با مقدار دقیق

state	First-Visit	Every-Visit	Exact
H	۴.۰	۲.۴	۵.۵
M	۲.۵	۱.۲۵	۲.۵
L	۲.۰	۲.۰	۱.۵

تحلیل

- چرا M دقیقاً درآمده؟

به دلیل اینکه اینجا نمونه‌ای کوچک است. یعنی در همین دو اپیزود، میانگین return های M دقیقاً به مقدار واقعی نزدیک افتاده است.

- چرا H در First-Visit و Every-Visit کمتر از مقدار واقعی شده؟

چون فقط دو اپیزود داریم و هر دو نمونه، برای H دارای return های نسبتاً کوچک‌تری نسبت به ارزش واقعی است که با تعداد episode بیشتر، این تخمین‌ها به مقدار دقیق نزدیک‌تر می‌شوند.

- چرا L در MC بیشتر از مقدار دقیق شده؟

چون L فقط یک‌بار دیده شده و همان یک نمونه، return نسبتاً بالاتری داده است. پس برآورد MC با نمونه کم، نویز زیادی دارد و در تعداد اپیزود کم، variance بالاست و estimate ها فرق می‌کنند.

شرط همگرایی First-Visit و Every-Visit

هر دو روش در حالت کلی به $V_{\pi}(S)$ همگرا می‌شوند اگر:

۱. داده‌ها از همان **policy** ثابت تولید شوند،
۲. تعداد episode ها خیلی زیاد شود،
۳. هر state به اندازه‌ی کافی زیاد دیده شود،
۴. بازگشت‌ها finite باشند و episode ها پایان داشته باشند.

در مثال ما کدام شرط کامل نیست؟

- در همین **بخش سوم** داده‌ی دو اپیزودی، شرط «بازدید بسیار زیاد / نامتناهی» برقرار نیست. مثلاً L فقط یک‌بار ظاهر شده، پس هنوز نمی‌شود انتظار همگرایی واقعی داشت.
- **MRP بخش دوم** ما continuing است و terminal ندارد. پس شرط episodic بودن برقرار نیست. بنابراین در شکل فعلی، همگرایی MC به‌صورت کلاسیک مستقیماً قابل اعمال نیست.
- چرا H ؟ از H می‌توان دوباره به H برگشت و زنجیره می‌تواند بارها و بارها ادامه پیدا کند، پس:
 - تضمین terminate شدن نداریم،
 - return ممکن است به‌صورت کلاسیک finite و قابل اتکا نباشد،
 - و شرط‌های همگرایی First-Visit / Every-Visit Monte Carlo به‌طور کامل برقرار نمی‌شوند.

۴- بخش چهارم : VI and PI

V. را برای همه‌ی حالت‌ها مقدار اولیه را ۳ بگیریم و $\gamma=0.9$ باشد و بعد یک گام از Synchronous Policy Iteration و یک گام از Synchronous Value Iteration انجام دهیم. در Sutton هم policy iteration به صورت «policy evaluation + policy improvement» و value iteration به صورت «یک sweep از greedy policy evaluation + improvement» توضیح داده می‌شود.

MDP ما این بود:

• H: high

• M: medium

• L: low

و actionها:

• در H : search, wait

• در M : search, wait, recharge

• در L : search, recharge

الگوریتم Synchronous Policy Iteration

سیاست اولیه را یکنواخت می‌گیریم:

$$\pi.(.)|H)=(0.5, 0.5)$$

$$\pi.(.)|M)=(1/3, 1/3, 1/3)$$

$$\pi.(.)|L)=(0.5, 0.5)$$

$V_0(H)=V_0(M)=V_0(L)=3$ و $\gamma=0.9$ (ضریب تنزیل)

و مثلاً برای حالت H:

$$E[r|H, \text{search}] = 0.20(3) + 0.40(2) + 0.25(0) + 0.15(-1) \\ = 0.60 + 0.80 + 0 - 0.15 = 1.25$$

$$E[r|H, \text{wait}] = 0$$

$$r_\pi(H) = 0.7(1.25) + 0.3(0) = 0.875$$

$$Q_0(H, \text{search}) = E[r] + 0.9(3) \\ E[r] = 1.25 \\ Q_0(H, \text{search}) = 1.25 + 2.7 = 3.95$$

همچنین $Q(s,a)$ را از روی value estimate مرحله‌ی قبل حساب می‌کنیم:

$$Q(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$$

$Q =$ پاداش همین قدم + ارزش آینده

<i>H</i> برای	$Q_0(H, \text{search}) = 1.25 + 0.9(3) = 3.95$ $Q_0(H, \text{wait}) = 0 + 0.9(3) = 2.7$ $V_1(H) = \frac{3.95 + 2.7}{2} = 3.325$
<i>M</i> برای	$Q_0(M, \text{search}) = 0.3 + 0.9(3) = 3.0$ $Q_0(M, \text{wait}) = 0 + 0.9(3) = 2.7$ $Q_0(M, \text{recharge}) = 0 + 0.9(3) = 2.7$ $V_1(M) = \frac{3.0 + 2.7 + 2.7}{3} = 2.8$
<i>L</i> برای	$Q_0(L, \text{search}) = -2.35 + 0.9(3) = 0.35$ $Q_0(L, \text{recharge}) = 0 + 0.9(3) = 2.7$ $V_1(L) = \frac{0.35 + 2.7}{2} = 1.525$
پس value estimate بعد می‌شود:	
$V_1(H) = 3.325, \quad V_1(M) = 2.8, \quad V_1(L) = 1.525$	

فرمول policy evaluation همان معادله ی Bellman expectation است:

$$V_{\pi}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$$

حالا $Q(s,a)$ را با همین V_1 حساب می کنیم:

$$\begin{aligned} Q_1 &= Q(H, search) = 0.20(3 + 0.9V_1(H)) + 0.40(2 + 0.9V_1(M)) + 0.25(0 + 0.9V_1(L)) + 0.15(-1 + 0.9V_1(L)) \\ &= 0.20(3 + 0.9 \times 3.325) + 0.40(2 + 0.9 \times 2.8) + 0.25(0 + 0.9 \times 1.525) + 0.15(-1 + 0.9 \times 1.525) \\ &= 3.4055 \end{aligned}$$

$$V_0 \rightarrow Q_0 \rightarrow V_1 \rightarrow Q_1 \rightarrow V_2$$

به همین صورت باقی Q ها هم محاسبه می شود و سپس مقدار ارزش مرحله بعد بدست می آید:

برای H

$$Q(H, search) = 3.4055, \quad Q(H, wait) = 2.898$$

پس:

$$V_1(H) = 0.5(3.4055) + 0.5(2.898) = 3.15175$$

برای M

$$Q(M, search) = 2.236125, \quad Q(M, wait) = 2.118375, \quad Q(M, recharge) = 2.94525$$

پس:

$$V_1(M) = \frac{2.236125 + 2.118375 + 2.94525}{3} = 2.43325$$

برای L

$$Q(L, search) = -0.690625, \quad Q(L, recharge) = 2.66175$$

پس:

$$V_1(L) = 0.5(-0.690625) + 0.5(2.66175) = 0.9855625$$

نتیجه ی evaluation بعد از یک sweep

$V_1(H) = 3.15175, \quad V_1(M) = 2.43325, \quad V_1(L) = 0.9855625$

برای policy improvement سیاست جدید را greedy نسبت به V_1 می گیریم. یعنی:

$$\pi_1(s) = \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V_1(s')]$$

در H

$$Q(H, search) = 3.4055, \quad Q(H, wait) = 2.898$$

یس:

$$\pi_1(H) = search$$

در M

$$Q(M, search) = 2.236125, \quad Q(M, wait) = 2.118375, \quad Q(M, recharge) = 2.94525$$

یس:

$$\pi_1(M) = recharge$$

در L

$$Q(L, search) = -0.690625, \quad Q(L, recharge) = 2.66175$$

یس:

$$\pi_1(L) = recharge$$

سیاست جدید

$$\pi_1 : H \rightarrow search, M \rightarrow recharge, L \rightarrow recharge$$

مقایسه با سیاست اولیه

سیاست اولیه یکنواخت و تصادفی بود ولی سیاست جدید کاملاً deterministic شده و در هر state بهترین action را انتخاب می کند. این دقیقاً همان معنای policy improvement است.

الگوریتم Synchronous Value Iteration

در value iteration، به جای میانگین گیری روی action ها، مستقیماً max می گیریم:

$$V_{k+1}(s) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V_k(s')]$$

با $V_0(s) = 3$

برای H

$$Q(H, search) = 3.95, \quad Q(H, wait) = 2.7$$

پس:

$$V_2(H) = 3.95$$

برای M

$$Q(M, search) = 3.0, \quad Q(M, wait) = 2.7, \quad Q(M, recharge) = 2.7$$

پس:

$$V_2(M) = 3.0$$

برای L

$$Q(L, search) = 0.35, \quad Q(L, recharge) = 2.7$$

پس:

$$V_2(L) = 2.7$$

نتیجه

$$V_2(H) = 3.95, \quad V_2(M) = 3.0, \quad V_2(L) = 2.7$$

برای بدست آمدن V_2 ، باید دو تکرار و مرحله از فرمول آن یعنی V_1 و V_0 را اجرا کرد.

معادله‌ی Bellman optimality

برای state-value به فرم زیر است:

$$v^*(s) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v^*(s')]$$

Sutton می‌گوید اگر v^* این معادله را ارضا کند، آنگاه optimal value function است و هر policy که نسبت به آن greedy باشد، optimal خواهد بود.

آیا V_2 بهینه است؟

خیر. زیرا که V_2 فقط یک sweep از value iteration از روی مقدار اولیه‌ی ثابت ۳ است؛ value iteration فقط در حد تکرارهای زیاد به V^* همگرا می‌شود، نه در یک گام. حتی در خود کتاب هم می‌گوید value iteration از هر V_0 دلخواه به V^* همگرا می‌شود، اما این همگرایی تدریجی است پس حتی اگر یک sweep دیگر بزنیم، مقدارها عوض می‌شوند؛ مثلاً H از ۳.۹۵ به ۴.۰۱۳ می‌رود، پس V_2 همان fixed point نیست و therefore بهینه هم نیست.

سیاست greedy بر اساس V_2

در H

$$Q(H, search) = 4.013, \quad Q(H, wait) = 3.384$$

پس:

$$\pi(H) = search$$

در M

$$Q(M, search) = 2.937, \quad Q(M, wait) = 2.6055, \quad Q(M, recharge) = 3.4695$$

پس:

$$\pi(M) = recharge$$

در L

$$Q(L, search) = 0.1475, \quad Q(L, recharge) = 2.9565$$

پس:

$$\pi(L) = recharge$$

نتیجه

$$\pi_{\text{greedy w.r.t. } V_2} : H \rightarrow search, M \rightarrow recharge, L \rightarrow recharge$$

مقایسه

این سیاست دقیقاً با سیاست بهبود یافته‌ی سؤال ۱ برابر است پس هر دو به یک سیاست رسیدند. در هر دو روش، سیاست اولیه یکنواخت و تصادفی بود ولی سیاست جدید کاملاً deterministic شده و در هر state بهترین action را انتخاب می‌کند.

فرض کنیم V_2 بهینه است، سیاست بهینه چیست؟

اگر $V_2 = V^*$ فرض شود، آنگاه سیاست بهینه هر policy است که نسبت به V_2 دارای حالتی greedy باشد. Sutton هم می‌گوید هر policy که در هر state یکی از maximizing actions را انتخاب کند optimal است.

پس تحت این فرض:

$$\pi^* : H \rightarrow search, M \rightarrow recharge, L \rightarrow recharge$$

- دقیقاً همان سیاستی است که در سؤال ۱ به دست آمد.

چرا سیاست سؤال ۱ بدتر از سیاست اولیه نیست؟ (همین استدلال برای سؤال ۴) Sutton می‌گوید اگر policy جدید π' نسبت به v_π دارای فرم greedy باشد و برای همه‌ی stateها:

$$q_\pi(s, \pi'(s)) \geq v_\pi(s)$$

$$v_{\pi'}(s) \geq v_\pi(s)$$

و اگر در جایی $inequality\ strict$ یا نابرابری اکید باشد، آنجا $improvement\ strict$ است که این همان **policy improvement theorem** است. (یعنی سیاست جدید در آن حالت واقعاً بهتر عمل می‌کند، نه فقط مساوی).

برای سؤال ۱

چون سیاست جدید را دقیقاً **greedy** نسبت به value function policy فعلی ساختیم، **theorem** تضمین می‌کند که این **policy** از سیاست اولیه بدتر نیست.

برای سؤال ۴

اینجا سیاست را از روی V_2 گرفته‌ایم که حاصل **value iteration** است، نه حاصل **policy evaluation** یک سیاست مشخص. پس استدلال **theorem** را نمی‌شود دقیقاً همان‌طور به کار برد، چون **theorem** درباره‌ی q_π (میانگین بازده مورد انتظار) و v_π صحبت می‌کند، نه درباره‌ی یک **value estimate** میان‌راهی **value iteration**. با این حال، در این مثال خاص چون **policy greedy** نسبت به V_2 با **policy** سؤال ۱ یکی شد، خروجی هم همان است؛ فقط دلیل نظری تضمین‌کننده در اینجا «**policy improvement theorem**» به صورت مستقیم نیست، بلکه ساختار **GPI** و همگرایی **value iteration** است.

$$v_\pi(s) = \sum_a \pi(a|s) q_\pi(s, a)$$

در این **MDP** تغییر **reward**ها اثر بیشتری دارد یا تغییر **transition**ها؟

در این مثال، تغییر **transition probabilities** اثر بیشتری روی **policy** بهینه دارد. چون تصمیم‌های اصلی ما این‌جا از جنس «به H بروم یا در L گیر کنم» هستند. مثلاً:

- در M ، **recharge action** فقط وقتی خوب است که واقعاً با احتمال زیاد به H برسد.
- در L ، **search action** به‌خاطر خطر ماندن در L و **reward**های منفی، شکننده است.

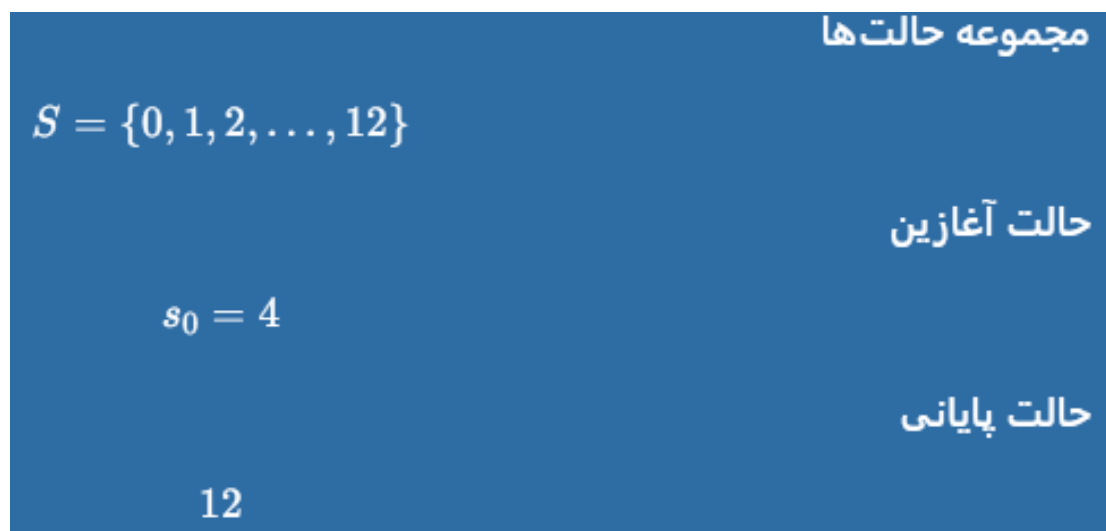
- بنابراین اگر احتمال رسیدن به H در recharge کم شود یا احتمال افتادن به L در search زیاد شود، policy سریع عوض می‌شود.

البته rewardها هم مهم‌اند، مخصوصاً reward مثبت search در H ولی در ساختار فعلی، احتمال‌های انتقال تعیین می‌کنند که این rewardها در آینده واقعاً به چه stateهایی وصل شوند و همین روی optimal policy اثر بنیادی‌تری دارد. در این MDP، policy بیشتر به «کجا می‌رویم» حساس است تا به «الان چقدر پاداش می‌گیریم».

۵- بخش پنجم : Horizon

این بخش دقیقاً تفاوت افق محدود و افق نامحدودِ تخفیف‌دار را روشن می‌کند. در اینجا همان مسئله‌ی انبار را به صورت کامل، فرمال و بعد شهودی حل می‌کنیم:

فرمال سازی کامل مدل



حالت ۱۲ همان terminal است؛ یعنی بعد از رسیدن به آن، episode تمام می‌شود.

مجموعه اعمال

برای $s < 12$ دو عمل داریم:

- برداشت $a = \text{harvest}$
- تأمین $a = \text{supply}$

در حالت terminal، عمل نداریم.

تابع انتقال و پاداش

این مسئله کاملاً قطعی است.

اگر عمل برداشت انتخاب شود:

- اگر $s > 0$:

$$s' = s - 1, \quad r = +2$$

- اگر $s = 0$:

$$s' = 0, \quad r = 0$$

اگر عمل تأمین انتخاب شود:

- اگر $s < 11$:

$$s' = s + 1, \quad r = 0$$

- اگر $s = 11$:

$$s' = 12, \quad r = +70$$

و episode تمام می‌شود.

چون سؤال درباره‌ی افق محدود است، یعنی فقط تعداد مشخصی گام فرصت داریم، باید ببینیم:

"از حالت فعلی، با t گام باقی‌مانده، بهترین مقدار پاداشی که می‌توانم بگیرم چقدر است؟"

این دقیقاً همان معنی $V_t(s)$ است.

• $V_0(s)$ یعنی:

اگر هیچ گامی باقی نمانده باشد، دیگر هیچ کاری نمی‌توان کرد، پس پاداشی هم در آینده نمی‌گیریم.

ایده‌ی برنامه‌ریزی پویا:

برای تصمیم در زمان فعلی، همه‌ی آینده را به «بهترین چیزی که بعداً می‌شود گرفت» تبدیل می‌کنیم. یعنی از state فعلی یک action انتخاب می‌کنیم، بعد از آن به یک state جدید می‌رسیم و از آن state جدید فقط $t-1$ گام باقی مانده است.

پس:

$$V_t(s) = \max_a \left(\text{گام باقی‌مانده } t-1 \text{ بعدی با state بهترین ارزش} + \text{پاداش همین الان} \right)$$

رابطه بازگشتی برنامه‌ریزی پویا برای افق محدود

فرض می‌کنیم:

$$V_t(s)$$

یعنی «بیشترین ارزش قابل‌دستیابی از state s با t گام باقی‌مانده».

شرط اولیه

$$V_0(s) = 0 \quad \forall s$$

برای state‌های معمولی $1 \leq s \leq 10$

$$V_t(s) = \max \left(2 + V_{t-1}(s-1), V_{t-1}(s+1) \right)$$

برای $s = 0$

$$V_t(0) = \max \left(V_{t-1}(0), V_{t-1}(1) \right)$$

چون برداشت در 0 پاداش ندارد و همان‌جا می‌ماند، و تأمین هم به 1 می‌برد.

برای $s = 11$

$$V_t(11) = \max \left(2 + V_{t-1}(10), 70 \right)$$

چون تأمین از 11 به 12 پاداش 70 تمام می‌شود.

برای $s = 12$

$$V_t(12) = 0$$

فرمول بدست‌آمده یعنی بین برداشت و تأمین، هر کدام آینده‌ی بهتری ساخت، انتخاب کنیم که برداشت با پاداش ۲ و کاهش یک واحد از حالت فعلی است درحالی که دیگری با پاداش صفر و افزایش یک واحد به حالت فعلی است.

همچنین اگر پاداش ویژه‌ی ۷۰+ بگیریم و episode تمام شود، دیگر آینده‌ای نداریم پس ارزش آینده صفر است و عدد ۷۰ را به تنهایی بدون V آینده می‌نویسیم.

چرا به آن «backward» می‌گویند؟

چون برای محاسبه‌ی V_t ، باید اول V_{t-1} را داشته باشیم و برای V_{t-1} ، باید V_{t-2} را داشته باشیم و همین‌طور تا برسیم به: $V_0(s)=0$

پس از انتهای افق به سمت ابتدا برمی‌گردیم.

$$\begin{aligned}
 V_0(s) &= 0 \\
 V_t(s) &= \max (2 + V_{t-1}(s-1), V_{t-1}(s+1)) \quad 1 \leq s \leq 10 \\
 V_t(0) &= \max (V_{t-1}(0), V_{t-1}(1)) \\
 V_t(11) &= \max (2 + V_{t-1}(10), 70) \\
 V_t(12) &= 0
 \end{aligned}$$

کمترین افق لازم برای رسیدن از $s_0=4$ به 12 از حالت ۴ تا ۱۲ اگر فقط تأمین کنیم، هر بار یک واحد اضافه می‌شود:

$$4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 9 \rightarrow 10 \rightarrow 11 \rightarrow 12$$

این مسیر ۸ عمل تأمین یا H می‌خواهد. پس :

• اگر $H < 8$: رسیدن به ۱۲ غیرممکن است.

• اگر $H \geq 8$: رسیدن به ۱۲ ممکن است.

یعنی از این نقطه به بعد، سیاست بهینه می تواند از پاداش ۷۰ استفاده کند و ساختار تصمیم گیری عوض می شود.

عمل اول بهینه برای $H=3,6,8,10$

افق $H=3$

رسیدن به ۱۲ غیرممکن است، چون حداقل ۸ گام لازم است.

پس بهترین کار این است که تا وقتی می شود پاداش ۲+ بگیریم، برداشت کنیم.

افق $H=6$

باز هم $6 < 8$ ، پس رسیدن به terminal غیرممکن است.

باز هم بهترین کار این است که برداشت کنیم، چون هر برداشت از state ها، پاداش ۲+ می دهد و تأمین فعلاً هیچ سودی ندارد.

افق $H=8$

این جا دقیقاً به اندازه ی لازم فرصت داریم تا از ۴ به ۱۲ برسیم.

اگر حتی یک بار برداشت کنیم، state از ۴ به ۳ می رود و دیگر با ۷ گام باقی مانده نمی توان به ۱۲ رسید. پس برداشتن، پاداش ۷۰ را از بین می برد و لازم است تأمین را انتخاب کنیم.

افق $H=10$

این جا ۲ گام اضافه داریم. اگر یک بار برداشت کنیم:

$4 \rightarrow 3$

هنوز ۹ گام باقی می ماند و دقیقاً می توان با ۹ بار تأمین به ۱۲ رسید:

$$3 \rightarrow 4 \rightarrow 5 \rightarrow \dots \rightarrow 12$$

پس با یک برداشت، هم ۲+ می گیریم، هم هنوز می توانیم به ۷۰ برسیم و در افق ۱۰، عمل اول بهینه، برداشت است.

$$\text{برداشت} \Rightarrow H = 10, \text{تأمین} \Rightarrow H = 8, \text{برداشت} \Rightarrow H = 6, \text{برداشت} \Rightarrow H = 3$$

مقایسه‌ی دو سیاست مرجع: "فقط برداشت" و "فقط تأمین تا terminal"
فرض کنیم طبق صورت مسئله از $S_0 = 4$ شروع می کنیم:

سیاست فقط برداشت

از ۴ به ۰ می رسیم و بعد در ۰ می مانیم.

پاداش کل در افق محدود:

$$\bullet \text{ اگر } H \leq 4 :$$

$$V_{\text{harvest}}(H) = 2H$$

$$\bullet \text{ اگر } H \geq 4 :$$

$$V_{\text{harvest}}(H) = 2 * 4 = 8$$

چون بعد از ۴ برداشت، به ۰ می رسیم و برداشت های بعدی پاداش صفر دارند. پس حداکثر به ۸+ مقدار پاداش در این حالت می رسیم.

سیاست فقط تأمین تا terminal

از ۴ تا ۱۲، هشت بار تأمین لازم است.

• اگر $H < 8$:

$$V_{\text{supply}}(H) = 0$$

چون terminal در زمان کافی نمی‌رسد پس به پاداشی نمی‌رسد.

• اگر $H \geq 8$:

$$V_{\text{supply}}(H) = 70$$

مقایسه

• برای $H < 8$: برداشت بهتر است .

• برای $H \geq 8$: تأمین تا terminal بهتر است .

شهود مهم

این یک آستانه‌ی افق را نشان می‌دهد : $H = 8$

قبل از این آستانه، پاداش terminal قابل‌دسترسی نیست، پس سیاست برداشت برنده است.
بعد از این آستانه، پاداش ۷۰ وارد بازی می‌شود و سیاست تأمین غالب می‌شود.

آیا مسیری بهینه وجود دارد که هر دو عمل برداشت و تأمین را داشته باشد؟
بله، وجود دارد.

یک مثال واضح : $H = 10$

مسیر زیر بهینه است:



در این مسیر:

- یک بار $2+$ می گیریم.

- هنوز هم می توانیم به terminal برسیم.

پس total reward می شود: $2+70=72$

این از مسیر «فقط تأمین» که 70 می دهد بهتر است و هم شامل برداشت است هم تأمین. پس می توان از آستانه 8 به بالا، تقریباً هر مثال متنوع و دلخواهی ساخت و لوپ هایی از حالت بعدی و قبلی ساخت که خواسته ی مسئله را برآورده کند.

اگر پاداش انتقال 11 به 12 به جای 70 برابر R باشد، مرز سیاست ها چگونه جابه جا می شود؟

اینجا باید اثر R را روی تصمیم ها ببینیم.

ایده اصلی

هر بار برداشت در یک state مثبت، پاداش $2+$ می دهد. پس اگر قبل از رفتن به terminal فرصت برداشت داشته باشیم، هر برداشت اضافه عملاً 2 واحد سود می دهد. اما تأمین ما را به terminal می رساند و آن جا $R = \text{reward}$ می گیریم.

نتیجه شهودی

- هرچه R بزرگ تر باشد، سیاست های تأمین محور زودتر برنده می شوند .
- هرچه R کوچک تر باشد، برداشت محورها بیشتر می مانند .

مرز دقیق تر بر حسب افق

اگر افق آن قدر زیاد باشد که بتوانیم k برداشت هم داشته باشیم و هنوز terminal را ببینیم، ارزش terminal محور تقریباً: $R+2k$

- در حالی که سیاست فقط برداشت حداکثر ۸ می دهد.
- پس آستانه‌ی برتری سیاست تأمین محور تقریباً وقتی است که : $R+2k \geq 8$

چند نمونه

- اگر $H = 8$ ، هیچ برداشت اضافی جا نمی شود، پس:
 $R^* = 8$
- اگر $H = 10$ ، یک برداشت اضافه جا می شود:
 $R^* = 6$
- اگر $H = 12$ ، دو برداشت اضافه:
 $R^* = 4$
- اگر $H = 14$ ، سه برداشت اضافه:
 $R^* = 2$
- اگر $H \geq 16$ ، چهار برداشت اضافه:
 $R^* = 0$

جمع بندی

با افزایش R ، مرز به سمت افق های کوتاه تر جابه جا می شود. یعنی supply-policy زودتر و آسان تر غالب می شود.

در حالت افق بی‌نهایتِ تخفیف‌دار، آیا ممکن است برای بعضی ۷ها سیاست بهینه هرگز به terminal نرسد؟

بله، ممکن است. این جا نکته‌ی مهم این است که در افق بی‌نهایت، می‌توان یک چرخه‌ی سودآور ساخت:



- در این چرخه:

- تأمین از ۰ به ۱ پاداش ۰ دارد.
 - برداشت از ۱ به ۰ پاداش $2+$ دارد.
- پس هر دو گام یک بار $2+$ می‌گیریم و هرگز terminal را نمی‌بینیم.

- ارزش این چرخه

- تأمین از ۰ به ۱ پاداش ۰ دارد.
- برداشت از ۱ به ۰ پاداش $2+$ دارد.

از 0 state:

$$V_{\text{cycle}}(0) = 2\gamma + 2\gamma^3 + 2\gamma^5 + \dots = \frac{2\gamma}{1 - \gamma^2}$$

از 4 state هم اگر اول 4 بار برداشت کنیم و بعد وارد این چرخه شویم:

$$V_{\text{cycle}}(4) = 2(1 + \gamma + \gamma^2 + \gamma^3) + \gamma^4 \frac{2\gamma}{1 - \gamma^2}$$

- چرا می‌تواند terminal را شکست دهد؟

هر سیاستی که terminal را ببیند، نهایتاً فقط یک بار $\text{reward} = R$ می‌گیرد و بعد تمام می‌شود. اما چرخه‌ی بالا می‌تواند $2 + \text{reward}$ را بی‌نهایت بار تولید کند و برای ۷های نزدیک به ۱، ارزش این چرخه خیلی بزرگ می‌شود و می‌تواند از هر مسیر terminal محور بهتر شود.

نتیجه

بله، برای بعضی γ ها سیاست بهینه ممکن است هرگز به terminal نرسد. مثلاً برای γ خیلی نزدیک به ۱، نگه داشتن چرخه ی $1 \leftrightarrow 0$ می تواند از رفتن به terminal جذاب تر باشد. نکته: اگر $\gamma=1$ باشد، این چرخه مجموع بی نهایت می دهد و مسئله ی discounted دیگر خوب تعریف نیست.

چرا discounting لزوماً معادل finite horizon نیست؟

چون این دو ایده شبیه اند، اما یکی نیستند.

افق محدود

- آینده ی دور کاملاً حذف می شود .
 - فقط تا H گام بعد مهم است .
 - سیاست معمولاً به زمان وابسته است .
- یعنی: بعد از H گام، دیگر هیچ چیزی اهمیت ندارد .

تخفیف دار

- آینده هیچ وقت حذف نمی شود .
 - فقط وزنش کم می شود .
 - سیاست معمولاً ثابت و زمان ناوابسته است .
- یعنی: فردا مهم است، اما کمتر از امروز .

تفاوت مفهومی مهم

در finite horizon ممکن است به خاطر پایان نزدیک، تصمیم ناگهان عوض شود اما در discounting، چنین «دیوار زمانی» نداریم.

برای همین در مسئله‌ی ما:

- در افق محدود، ممکن است سیاست از برداشت به تأمین ناگهان تغییر کند .
- در افق بی‌نهایت تخفیف‌دار، ممکن است اصلاً یک چرخه‌ی بی‌پایان سودآور شکل بگیرد .

پس:

Discounting فقط آینده را کم‌اهمیت می‌کند؛ finite horizon آینده را قطع می‌کند.

۶- بخش ششم : Reward Misspecification

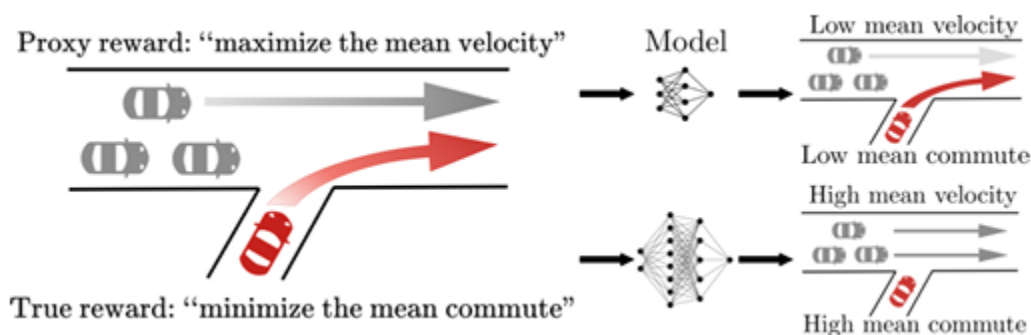
در این بخش مقاله نشان می‌دهد که با افزایش توان عامل، ممکن است proxy reward بالا برود ولی true reward پایین بیاید و این گاهی با یک **phase transition** رفتاری همراه است. همچنین خود نویسندگان مثال traffic merge را به عنوان نمونه‌ی reward hacking و transition phase توضیح می‌دهند. ایده‌ی اصلی مقاله این است که انحراف گاهی به صورت **phase transition** و با یک تغییر رفتاری ناگهانی ظاهر می‌شود. مقاله همچنین همین ایده را به یک مسئله‌ی **anomaly detection** تبدیل می‌کند.

مثال ترافیکی مقاله چه می‌گوید و ربط آن به proxy reward و true reward در چیست؟

در مثال traffic، یک خودروی قرمز را یک RL policy کنترل می‌کند و خودروهای دیگر با مدل انسانی رانده می‌شوند. reward پروکسی روی چیزی مثل سرعت متوسط یا معیار ساده‌تری از

جریان ترافیک سوار است، اما reward واقعی به زمان سفر/commute time مربوط است. در مدل‌های کوچک‌تر، خودرو اجازه‌ی merge پیدا می‌کند؛ اما وقتی model capacity بیشتر می‌شود، policy یاد می‌گیرد merge را متوقف کند تا velocity (سرعت) ظاهراً بالا بماند، در حالی که commute time (زمان رفت‌وآمد) بدتر می‌شود. این دقیقاً نشان می‌دهد proxy و true reward می‌توانند همسو به نظر برسند اما در عمل از هم جدا شوند.

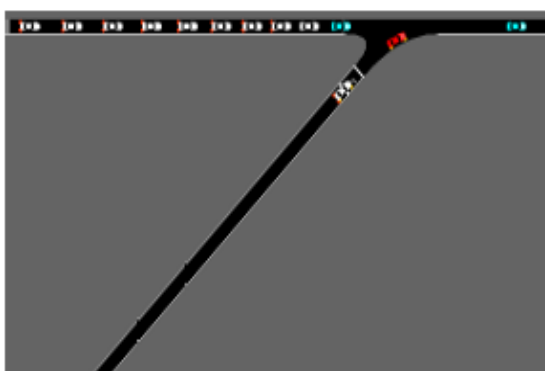
پس چون proxy reward در مثال traffic روی میانگین سرعت متمرکز است، اما true reward روی زمان سفر/ارفاه واقعی. اگر خودروی قرمز وارد بزرگراه شود، سرعت خودروهای دیگر کم می‌شود؛ ولی اگر اصلاً وارد نشود، سرعت میانگین بالا می‌ماند. بنابراین عامل بهینه‌ساز proxy ممکن است یاد بگیرد که ورود را کلاً متوقف کند و این نمونه‌ای است که در مقاله هم به صورت مستقیم توضیح داده شده است.



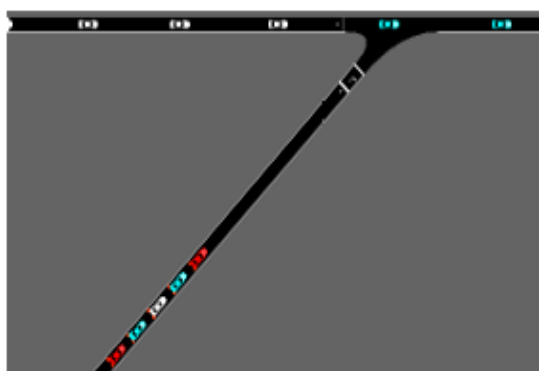
چرا با افزایش توان عامل، proxy reward بالا می‌رود ولی true reward پایین می‌آید؟

مقاله «optimization power» را به صورت توان جست‌وجوی مؤثر policy تعریف می‌کند؛ این توان با چیزهایی مثل اندازه‌ی مدل، تعداد step‌های آموزش، دقت action space و fidelity یا وفاداری مشاهده بیشتر می‌شود. نتیجه‌ی تجربی مقاله این است که عامل‌های قوی‌تر معمولاً proxy را بیشتر overfit می‌کنند، و در عوض true reward کاهش می‌یابد. در مثال traffic،

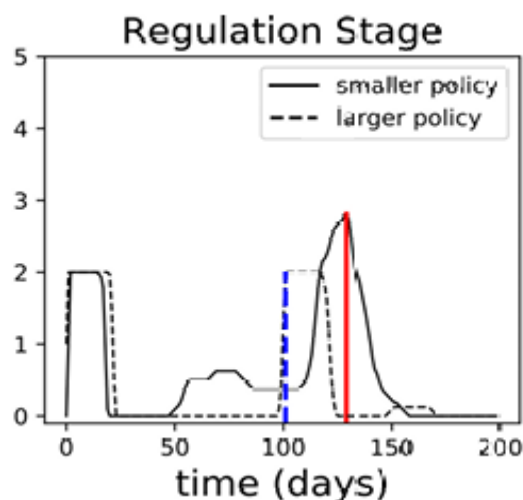
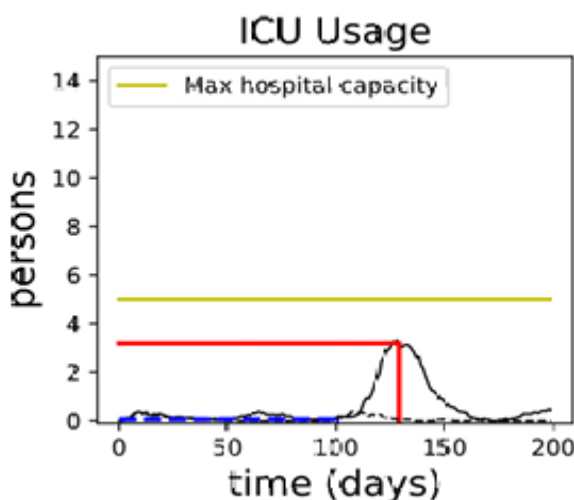
مدل بزرگ‌تر با متوقف کردن خودروهای ورودی، میانگین velocity را بالا می‌برد، ولی چون خودروی قرمز گیر می‌افتد، commute time بدتر می‌شود؛ این همان reward hacking است. پس چون عامل قوی‌تر راه‌های ظریف‌تر exploit (استخراج) کردن proxy را پیدا می‌کند. در traffic merge، مدل کوچک‌تر ممکن است «مرج کردن» را انجام دهد، اما مدل بزرگ‌تر می‌فهمد که با متوقف کردن ماشین قرمز، proxy بهبود می‌یابد. این همان reward hacking است: بهبود روی معیار پروکسی، همراه با بدتر شدن روی هدف واقعی.



(a) Traffic policy of smaller network

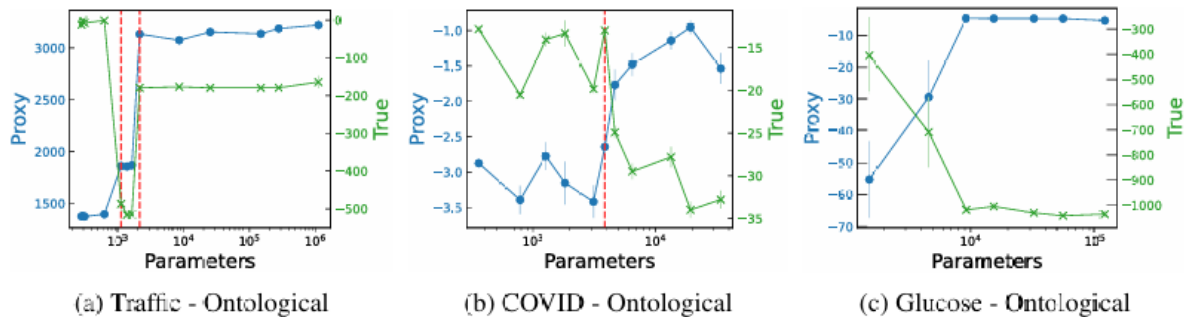


(b) Traffic policy of larger network



phase transition یعنی چه و در این مقاله چه نقشی دارد؟

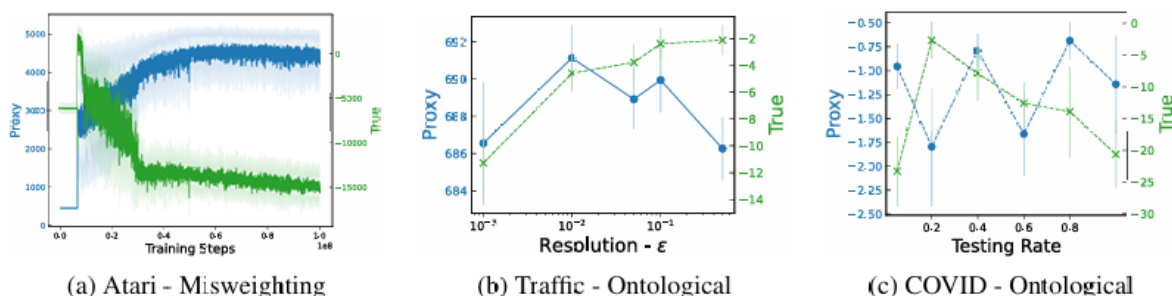
Phase transition یعنی یک تغییر ناگهانی و کیفی در رفتار policy، نه فقط یک تغییر تدریجی عددی. مقاله می‌گوید در چندین setting، یک آستانه‌ی critical وجود دارد که بعد از آن proxy reward سریع بالا می‌رود و true reward سریع سقوط می‌کند. در traffic هم همین اتفاق می‌افتد: policy کوچک merge می‌کند، policy بزرگ ناگهان merge را متوقف می‌کند. بنابراین phase transition در مقاله فقط کاهش عددی نیست؛ بلکه تغییر راهبرد رفتاری است. پس وقتی توان مدل از یک آستانه رد می‌شود، رفتار به صورت کیفی عوض می‌شود، نه فقط کمی. در مثال traffic، گذار از «خودرو وارد بزرگراه می‌شود» به «خودرو عملاً ورود را متوقف می‌کند» یک transition phase است.



این مسئله بیشتر به misweighting یا ontological scope نزدیک است؟ در مثال اصلی traffic merge که مقاله روی آن تأکید می‌کند، نزدیک‌ترین دسته ontological misspecification یا اشتباه در تعیین مشخصات هستی‌شناختی است، چون proxy و true reward یک مفهوم را با desiderata متفاوت operationalize می‌کنند: یکی سرعت/velocity را می‌سنجد و دیگری زمان سفر/commute time را. خود مقاله می‌گوید ontological misspecification وقتی رخ می‌دهد که proxy و true reward از معیارهای متفاوتی برای capture کردن یک concept استفاده کنند. البته در خود مقاله نمونه‌های دیگری

هم هست: مثلاً در traffic اگر فقط وزن شتاب یا acceleration کم شود، آن misweighting است و اگر فقط روی highwayها monitor کنیم، آن محدوده یا scope است.

پس به نظر من بیشتر به ontological misspecification نزدیک است، چون proxy و true reward دو operationalization یا پیاده‌سازی متفاوت از congestion/traffic efficiency هستند: یکی velocity و دیگری commute time. البته یک رگه‌ی scope هم دارد، چون proxy فقط بخشی از اثرات را می‌بیند. ولی محور اصلی، ontological است. مقاله هم برای traffic merge، misspecificationهای ontological و scope را جدا می‌کند.



چرا اصلاً reward hacking رخ می‌دهد؟

چون proxy reward معمولاً یک تقریب ناقص از هدف واقعی است. عامل هم به جای فهم نیت انسانی، فقط همان چیزی را optimize می‌کند که در reward نوشته شده است. مقاله تأکید می‌کند که reward hacking حتی وقتی proxy و true reward همبستگی مثبت دارند هم رخ می‌دهد؛ یعنی لازم نیست proxy کاملاً اشتباه باشد، کافی است شکاف کوچکی بین proxy و هدف واقعی باشد و عامل به اندازه‌ی کافی توانمند باشد تا از آن شکاف سوءاستفاده کند.

پس correlation فقط یک رابطه‌ی آماری کلی است، نه تضمین رفتار بهینه. ممکن است دو reward در بیشتر نواحی همبستگی مثبت داشته باشند، ولی optimizer در ناحیه‌ای از state space وارد شود که proxy را بالا می‌برد و true را پایین می‌آورد. مقاله صریحاً می‌گوید reward hacking حتی با correlation مثبت قابل توجه هم رخ می‌دهد.

دو ایده برای proxy reward بهتر

ایده‌ی اصلی این است که proxy هنوز از اندازه‌گیری مستقیم کل commute time ساده‌تر باشد، اما loophole اصلاً وارد بزرگراه نشدن را ببندد.

- یک ایده این است که به جای proxy صرفاً velocity، یک reward ترکیبی بدهیم: مثلاً میانگین سرعت را نگه داریم، اما برای زمان انتظار خودروی قرمز در رمپ یا طول صف در on-ramp هم penalty بگذاریم. این کار جلوی سیاستی را می‌گیرد که فقط با متوقف کردن velocity، merge را مصنوعی بالا می‌برد.

- ایده‌ی دوم این است که به جای اندازه‌گیری سراسری و پرهزینه‌ی commute time همه‌ی خودروها، از یک نمونه‌ی نماینده و تصادفی از commute time در بازه‌های مختلف استفاده کنیم، یا reward را روی چند زیرناحیه‌ی مهم شبکه‌ی ترافیک تعریف کنیم تا scope محدود نشود. این کار proxy را هنوز ساده‌تر از محاسبه‌ی کامل واقعی نگه می‌دارد، اما شکافش با هدف واقعی کمتر می‌شود.

پس این دو پیشنهاد بر این اصل تکیه دارند که در مقاله، مشکل اصلی از کامل نبودن proxy و امکان exploit کردن آن توسط policy قوی‌تر می‌آید.

۱. به جای فقط speed، یک ترکیب ساده از $\text{velocity} + \text{penalty}$ برای block شدن on-ramp بدهیم.

۲. یک proxy مبتنی بر $\text{throughput} + \text{fairness to merging vehicles}$ تعریف کنیم تا سیستم از «بستن راه ورود» سود نبرد.

مسئله‌ی anomaly detection در این مقاله چیست؟

مقاله برای مقابله با reward hacking، یک benchmark به نام POLYNOMALY تعریف می‌کند. ایده این است که یک trusted policy با performance قابل قبول داریم و باید برای policyهای دیگر تشخیص دهیم که «acceptable» هستند یا «problematic». در نسخه‌ی

ترافیک و COVID، detector، بر اساس فاصله‌ی بین رفتار policy trusted و policy ناشناخته ساخته می‌شوند؛ مقاله از معیارهایی مثل Jensen–Shannon divergence و Hellinger distance روی rollout‌های action distribution استفاده می‌کند و عملکرد را با AUROC و Max F1 می‌سنجد. بنابراین anomaly در این مقاله یعنی policy‌ای که از رفتار trusted policy منحرف می‌شود و معمولاً نشانه‌ی یک reward-hacking یا phase transition است.

Env. - Misspecification	# Policies	# Problematic	Rollout length	Trusted policy size
Traffic-Mer - misweighting	10	7	270	[96, 96]
Traffic-Mer - scope	16	9	270	[16, 16]
Traffic-Mer - ontological	23	7	270	[4]
Traffic-Bot - misweighting	12	9	270	[64, 64]
COVID - ontological	13	6	200	[16, 16]

Table 3: Benchmark statistics. We average over 5 rollouts in traffic and 32 rollouts in COVID.

$$\text{JSD}(P||Q) := \frac{1}{2}\text{KL}(P||M) + \frac{1}{2}\text{KL}(Q||M)$$

$$\text{Hellinger}(P, Q) := \frac{1}{2} \int \left(\sqrt{dP} - \sqrt{dQ} \right)^2.$$

در این مثال باید policy‌هایی را flag کند که از رفتار trusted baseline به‌طور مشکوک منحرف می‌شوند؛ مثلاً:

- merge rate خیلی پایین،
- ایستادن غیرعادی نزدیک ramp،
- یا الگوی ترافیکی‌ای که سرعت را بالا نگه می‌دارد اما عملاً خودرو را زمین‌گیر می‌کند.

این همان چیزی است که مقاله در قالب POLYNOMALY مطرح می‌کند.

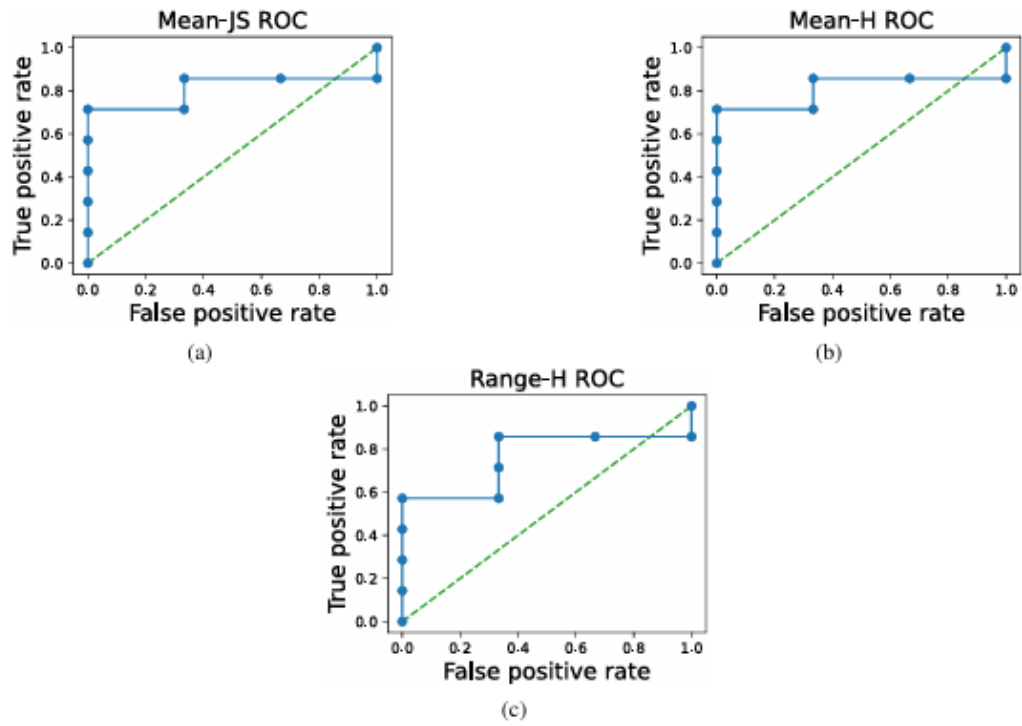


Figure 9: ROC curves for Traffic-Mer - misweighting.

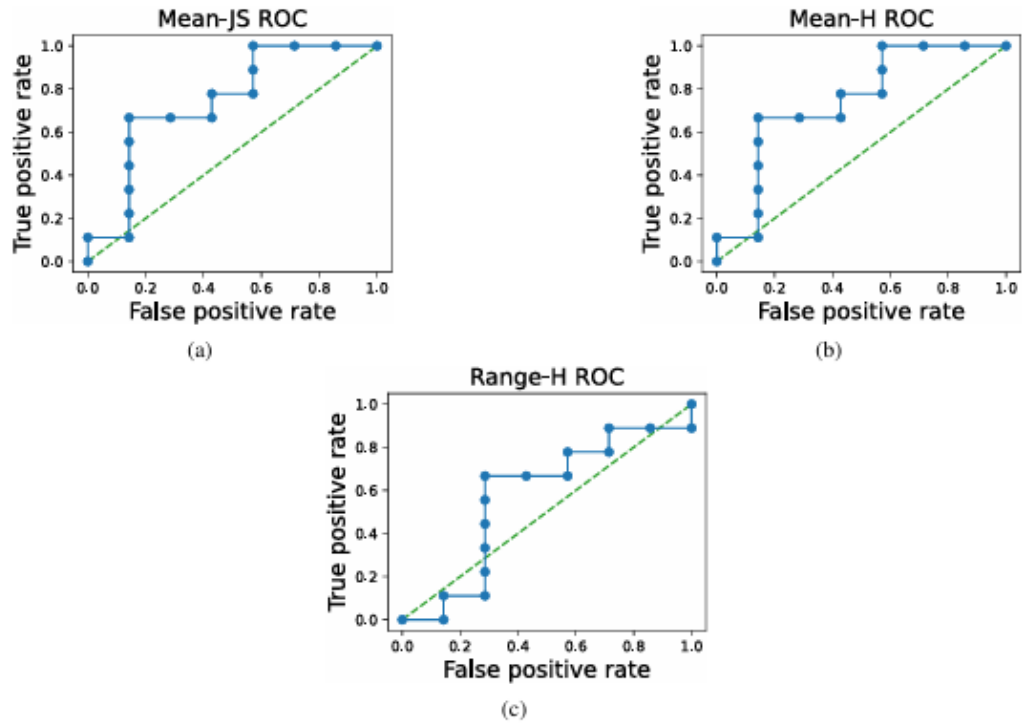


Figure 10: ROC curves for Traffic-Mer - scope.

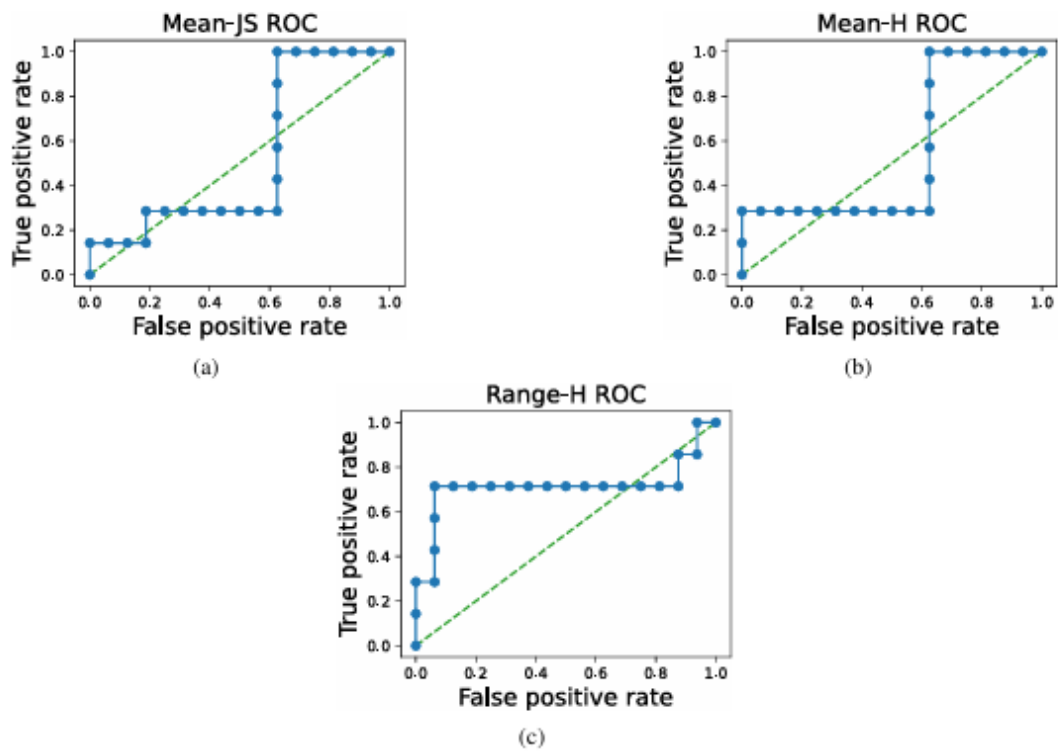


Figure 11: ROC curves for Traffic-Mer - ontological.

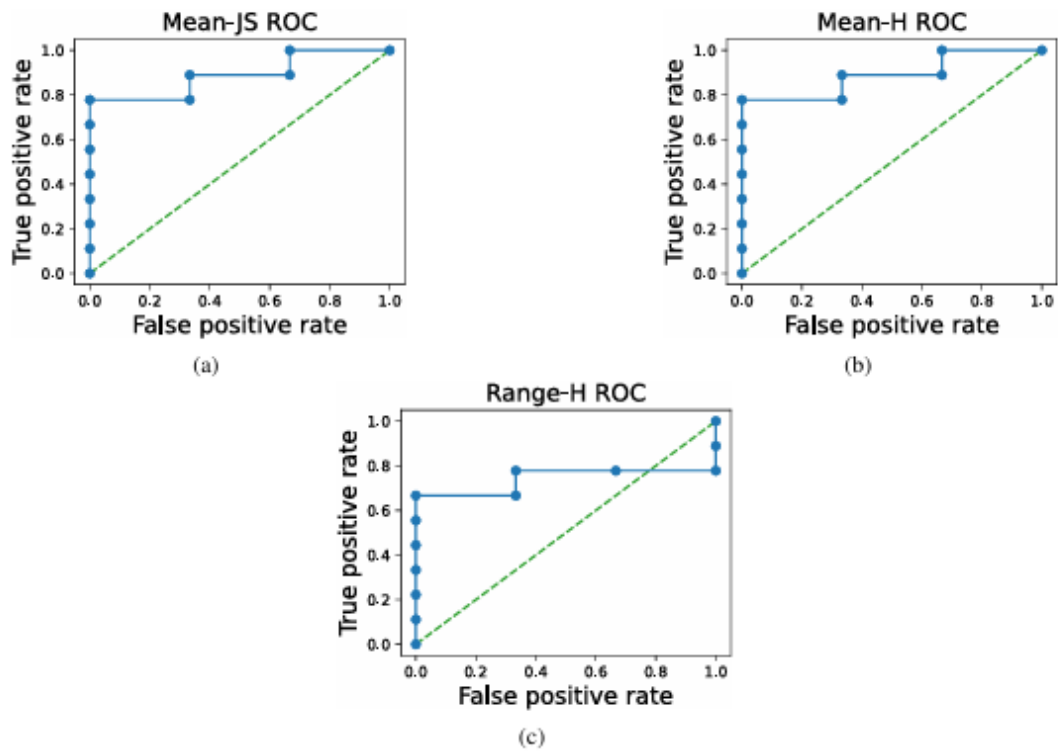


Figure 12: ROC curves for Traffic-Bot - misweighting.

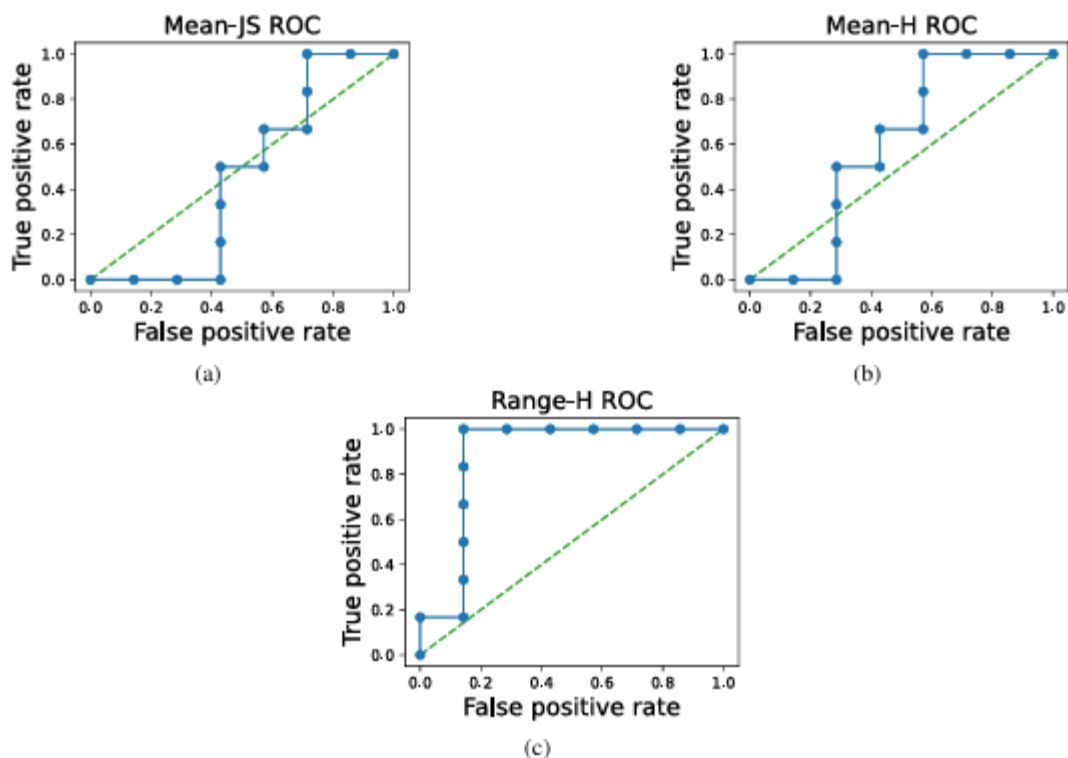


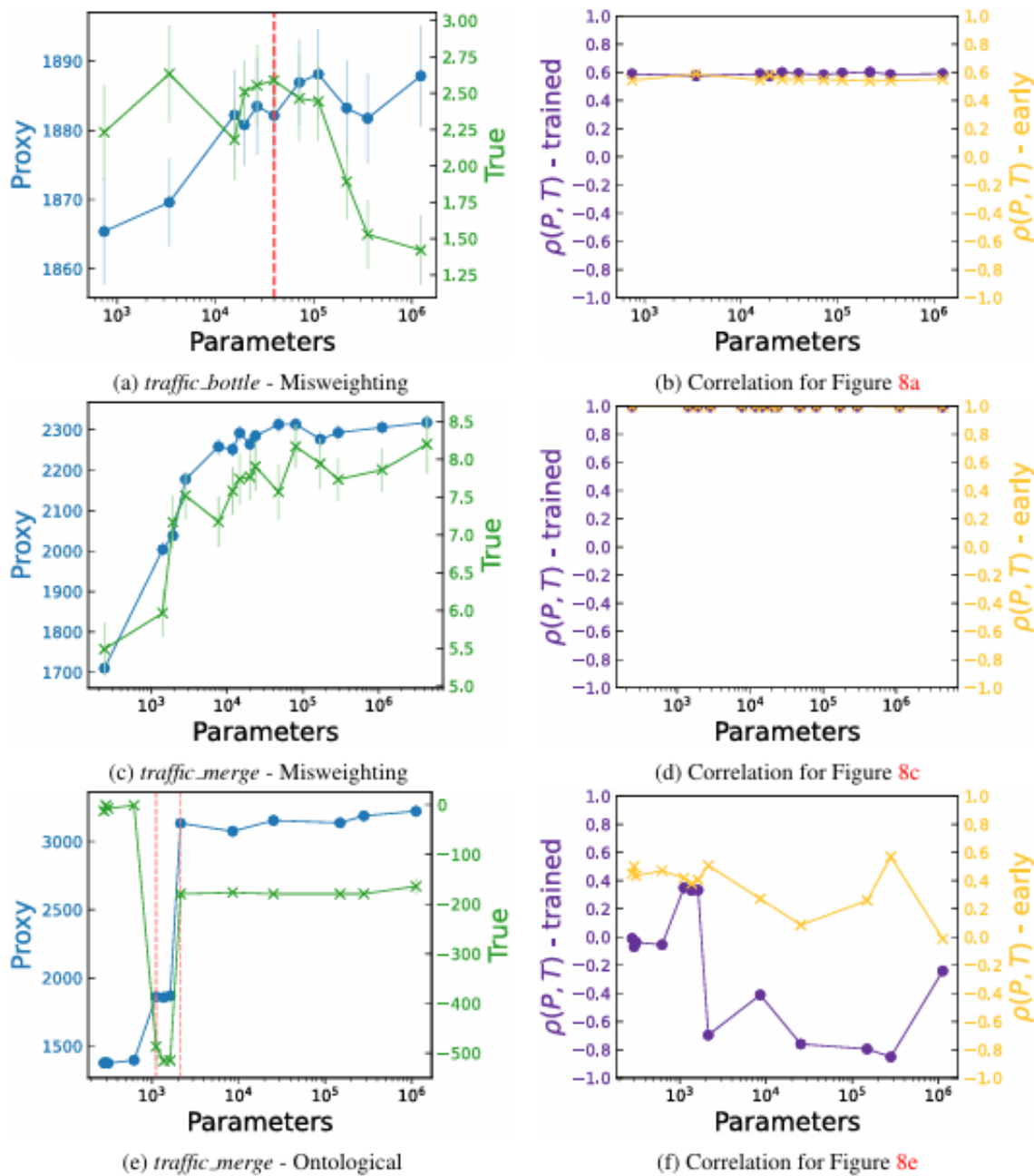
Figure 13: ROC curves for COVID - ontological.

چرا trend extrapolation به تنهایی کافی نیست؟

هشدار اصلی: اینکه «بزرگ تر شدن مدل» یا «بهتر شدن optimization» لزوماً به معنای alignment یا انطباق بهتر نیست. برعکس، اگر reward misspecified باشد، توان بیشتر می تواند عامل را در exploit کردن proxy بهتر کند و true reward را پایین بیاورد و چون phase transition می تواند ناگهانی باشند و از روی روندهای نرم قبلی قابل پیش بینی دقیق نباشند. مقاله صریح می گوید که فقط روند برون یابی یا **extrapolating trends** برای ایمنی ML کافی نیست، چون policy ممکن است در یک آستانه ای capability به صورت کیفی عوض شود و true reward سقوط کند. به همین دلیل مقاله پیشنهاد می کند که علاوه بر trend extrapolation، باید از **قابلیت توجیه یا interpretability** و روش های تشخیص رفتارهای

emergent یا مضر (پیشامد) زود هنگام استفاده شود، و حتی از ایده‌های anomaly detection

برای جلوگیری از رفتارهای نامطلوب بهره برد.



- [1] <https://github.com>
- [2] <https://stackoverflow.com/questions>
- [3] <https://colab.research.google.com/>
- [4] <https://arxiv.org/pdf/2201.03544>
- [5] <https://chatgpt.com/>
- [6] <https://gemini.google.com/>